QPU Benchmarking Framework Documentation

S. Louise

October 19, 2026

Contents

1 QPU Abstraction Framework for benchmarking 2

1.1 Overall Architecture . . . 3
1.2 BaseAbstractClass . . . 3
1.2.1 Base principle and implementation . . . 3
1.2.2 The `capabilitySet` list . . . 5
1.2.3 The `qpuDispatch` dictionary . . . 6
1.3 QPUBenchContext . . . 6
1.4 Concrete QPU Classes . . . 7
1.4.1 Implementation contract . . . 7
1.4.2 Typical file layout for one QPU . . . 7
1.5 Four-Step Instantiation Protocol . . . 9
1.6 Separation of Generic versus Specific Code . . . 9
1.7 Summary . . . 11
1.8 Short How-to: building a new support for a QPU . . . 11
1.8.1 Header . . . 12
1.8.2 Backend functions . . . 12
1.8.3 Concrete class implementation . . . 13
1.8.4 Hardware-Independent Sample Representation . . . 14
1.A BACQ Benchmark . . . 14
1.A.1 📋 Prerequisites . . . 15
1.A.2 🚀 Installation . . . 15
1.A.3 Basic unitary tests . . . 15
1.A.4 Usage . . . 15
1.A.5 To do . . . 16
1.B Doxygen documentation . . . 16

Chapter 1

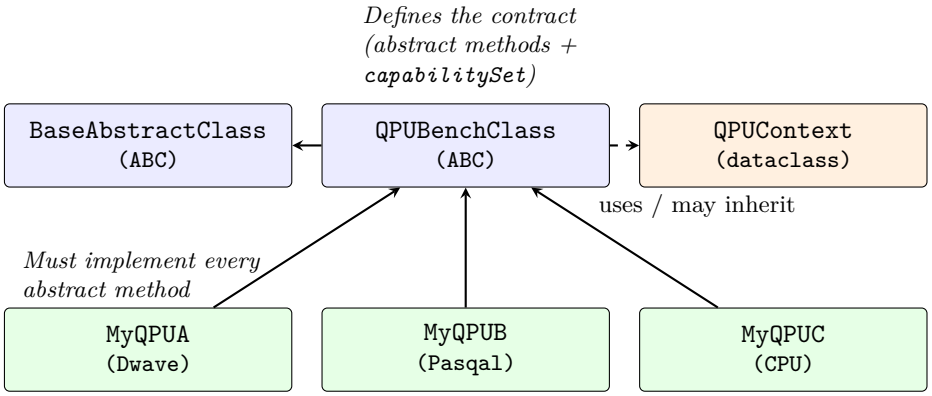
QPU Abstraction Framework for benchmarking

This chapter describes the core object-oriented architecture used throughout the Benchmarking framework and its libraries. The design centres on a single abstract base class that acts as a uniform canvas for every QPU implementation. Concrete QPU classes are realised by derivation, while a capability dictionary provides a completely target-independent call interface to the main program. Generic (QPU-independent) computational kernels live in dedicated library modules; only the truly platform-specific fragments reside inside the concrete QPU modules.

The presentation is deliberately dual-audience: UML-aware readers will recognise the classic Abstract-Base-Class / Template-Method pattern; readers unfamiliar with UML can simply follow the textual explanations and the accompanying diagrams.

The CPU hardware provides a compatible class that implements reference CPU algorithms and heuristics. This is particularly important to provide reference scores to compare classic computing and quantum computing.

1.1 Overall Architecture



- **QPUBenchClass** – the single canvas every QPU must respect.
- **Concrete classes** (`MyQPU_A`, `MyQPU_B`, ...) – one class per supported hardware target. The framework also provides a CPU backend for comparison between classical computing and quantum computing.
- **QPUContext** – a lightweight data container (Python `@dataclass`) that carries all values and intermediate structures between the generic framework and the QPU-specific code. Concrete QPUs may use it directly or derive a specialised context from it.

1.2 BaseAbstractClass

The abstract base class declares the mandatory interface and owns the central `capabilitySet` dictionary.

1.2.1 Base principle and implementation

```
from abc import ABC, abstractmethod
from dataclasses import dataclass
from typing import Dict, Callable, Any
import argparse

@dataclass
class QPUContext:
    """Carrier of all data exchanged between
       generic framework code and QPU-
       specific implementations."""
    # common fields (arrays, scalars, handles ...)
    ...

class QPUBenchClass(ABC):
```

```
15      @property
16      # In the property part, derived (concrete) classes
            must define:
17      # - capabilitySet -> list of strings: is the set of
            benchmark scores supported by the derived class
18      # - qpuDispatch -> dictionnary of list of function
            pointers. The key is the name of the benchmark
            that must be in the capabilitySet list
19      # - a QPUcontext to exchange the relevant data
            between functions and to the frontend
20      # - 3 function pointers preprocessMethod,
            computeMethod and postprocessMethod to call the
            relevant functions according to the bencmark
            score to run
21
22      @abstractmethod
23      def capabilities(self) -> QPUBenchClassCapability:
24          pass
25
26      @abstractmethod
27      def selectBenchmark(self, bench: str, **kwargs):
28          # generic method to set the function pointers
                self.preprocessMethod, self.computeMethod and
                 self.postprocessMethod
29          # the class methods below will make the actual
                call. The function pointers are allocated
                according to the selected benchmark
30          # it is worth noting that selectBenchmark method
                is (ATM) aimed to be called at the
                instatiation (by the constructor)
31          pass
32
33      @abstractmethod
34      def preprocess(self, data: Any, **kwargs) -> Any:
35          # method to call the self.preprocessMethod with
                the context
36          pass
37
38      @abstractmethod
39      def compute(self, data: Any, **kwargs) -> Any:
40          # method to call the self.computeMethod with the
                context
41          pass
42
43      @abstractmethod
44      def postprocess(self, data: Any, **kwargs) -> Any:
45          # method to call the self.postprocessMethod with
                the context
46          pass
```

```
47
48      @abstractmethod
49      def is_available(self) -> bool:
50          r"""Check if the QPU is available at runtime."""
51          pass
52
53      @classmethod
54      def add_arguments(cls, parser:argparse.ArgumentParser
            ):
55          r"""Generic class method to add arguments to the
                cli invocation depending on instancied class.
                 """
56          """Extend the global CLI with target-specific
                options (device id, layout, architecture,
                ...)."""
57          pass
58
59      # The three methods below are the concrete
            realisations
60      # of the three entries in capabilitySet.  They are
            still
61      # declared abstract so that every derived class is
            forced
62      # to supply an implementation.
63      @abstractmethod
64      def preprocess(self, ctx: QPUContext) -> None:
65          """Prepare the QPU and set its context."""
66          ...
67
68      @abstractmethod
69      def run(self, ctx: QPUContext) -> None:
70          """Execute the computation on the QPU."""
71          ...
72
73      @abstractmethod
74      def postprocess(self, ctx: QPUContext) -> Any:
75          """Post-process the result and return a
76              target-independent set of results."""
77          ...
```

Listing 1.1: Definition of QPUContext and BaseAbstractClass

1.2.2 The `capabilitySet` list

`capabilitySet` is a list of strings that provides all the benchmark scores currently supported by the concrete instance of QPUBenchClass. The benchmark names must be constistent for all the QPU and all the instances. The names of the benchmarks also provide the key to look in the dictionnary of function pointers `qpuDispatch` as described below.

1.2.3 The `qpuDispatch` dictionary

The dictionary `capabilitySet` is the sole communication channel used by the main program. It always contains exactly three function pointers:

Key	Responsibility
`"preprocess"`	Prepare the QPU (allocate buffers, transfer data, configure the device, ...) and finalise the `QPUContext`.
`"compute"`	Perform the actual computation on the QPU.
`"postprocess"`	Retrieve results from the device, free resources and return a completely target-independent set of results.

The main program never calls an QPU method by name. It only calls the generic methods that, in turn, invokes the corresponding function pointer:

```
1         qpu.preprocess()
2         qpu.compute()
3         benchdone, score= qpu.postprocess()
```

This keeps the whole framework completely independent of any particular QPU.

1.3 QPUBenchContext

The data-class QPUBenchClass is the main communication means between the concrete instance of a QPUBenchClass to support a given QPU on one hand and on the other hand, both the specific functions implementing e.g. Register allocation on the QPU (or setting the stage for the computation) and also the main program implemented from `BACQBenchmak.py`. The important aspects for this data class are:

- `nIter`: which provides the current interation of the current benchmark executing on the QPU. Note that this is the responsability of `BACQBenchmak` to increment this counter.
- `dat`: an optional secondary value to perform the given benchmark (e.g. number of run for each iteration). The responsability of this variable depends usually on the particular benchmark.
- `currentBench`: String value giving the normalized name of the benchmark within the program (the same which is used to look in the dictionnary of the class) (`BACQBenchmak`). It serves in order to attribute the correct function pointers in the dictionnary.
- `formatedSamples`: are the samples returned by the QPU after run. They maybe QPU dependent, but should serve as a base to communicate the results to the postprocess method. N.B.: the postprocess method return QPU independent formated samples.

For all practical purposes, it is possible to inherit this class to implement more relevant context management for a given QPU, but any inherited class should provide the same base as the generic class with the same semantic for the common parts.

1.4 Concrete QPU Classes

This section provides the principles of implementation of a QPU class inherited from `QPUBenchClass` to provide seamless support and inclusion into the multi-QPU benchmarking framework.

1.4.1 Implementation contract

Every concrete QPU class must:

1. inherit from `BaseQPUClass`,
2. implement *all* abstract methods (including `add_argument`, `preprocess`, `run` and `postprocess`),
3. bind the three life-cycle methods to the corresponding entries of `self.capabilitySet`.

QPU-specific helper functions that are not part of the public contract live in the same Python module, usually at the top of the file, outside the class body. Purely generic kernels (e.g. QUBO problem generation that does not depend specificly on the target) are placed in a separate computation library module and are imported by every concrete QPU that needs them.

1.4.2 Typical file layout for one QPU

Therefore, adding a new QPU to the set of supported QPU only consists in creating a concrete instance derivated from QPUBenchClass, on one hand, and adding the entries in the backend dictionary in the `BACQBenchmak.py` main program in the python dictionary `backendMap`.

Below, the base principle to implement a concrete QPU class.

```
1 # my_QPU_shiny.py
2
3 #
    ------------------------------------------------------------------------
4 # 1. QPU-specific low-level helpers
5 #
    ------------------------------------------------------------------------
6 def _qpu_register_alloc(...): ...
7 def _qpu_sample_read(...): ...
```

```
def _qpu_launch_kernel(...): ...

#
    ------------------------------------------------------------------

# 2. Concrete class
#
    ------------------------------------------------------------------

class MyQPU1(QPUBenchClass):
    capabilitySet= ['gscore', ...] # capabilitySet is
       required to support at least one benchmark score
    qpuDispatch= {'gscore':[qpuPreprocessGScore,
       qpuCompute, qpuPostprocessGScore], ...} # the
       dictionnary with the relevant function pointers
    QPUContext ctx

    def __init__(self, bench):
        super().__init__()
        # Bind the three mandatory capabilities (
           qpuDispatch is the dictionnary for the
           function pointers)
        selectBenchmark(bench)

    def selectBenchmark(self, bench: str, **kwargs):
        processSet= self.qpuDispatch[bench]
        self.myPreprocess= processSet[0]
        self.myCompute= processSet[1]
        self.myPostprocess= processSet[2]

    def add_argument(self, parser):
        parser.add_argument("--qpu1-device", type=sting,
           default='defaultarch')
        parser.add_argument("--qpu1-alg-select", ...)
        ...

    def preprocess(self, ctx: QPUContext):
        # allocate, copy host->device, set kernel
           parameters ...
        ...

    def run(self, ctx: QPUContext):
        # launch kernels
        ...

    def postprocess(self, ctx: QPUContext):
        # copy device->host, free resources,
        # return target-independent results
        return result
```

Listing 1.2: Example of a concrete QPU module

N.B.: In the current code the `capabilitySet` is rather defined statically in the class definition, rather than in the constructor.

1.5 Four-Step Instantiation Protocol

A concrete QPU is always used according to the same four-step protocol. The main program (or a higher-level driver) performs the following sequence:

1. **Construction** `qpu = MyQPU_A()` The constructor initialises internal state and populates `capabilitySet`.
2. **CLI extension** (optional but recommended) `qpu.add_argument(parser)` Target-specific command-line options are registered here.
3. **Pre-processing** After the main program has created a MyQPUClass instance with the data describing the requested computation, it calls `qpu.preprocess(ctx)`.
4. **Computation & post-processing**

```
1         qpu.compute(ctx)
2         done, result = qpu.postprocess(ctx)
```

The exact computation type is selected by setting appropriate fields inside the `QPUContext` (or a derived context) before the call to `preprocess`. The framework itself never hard-codes any QPU name; it only consults the `capabilitySet` dictionary.

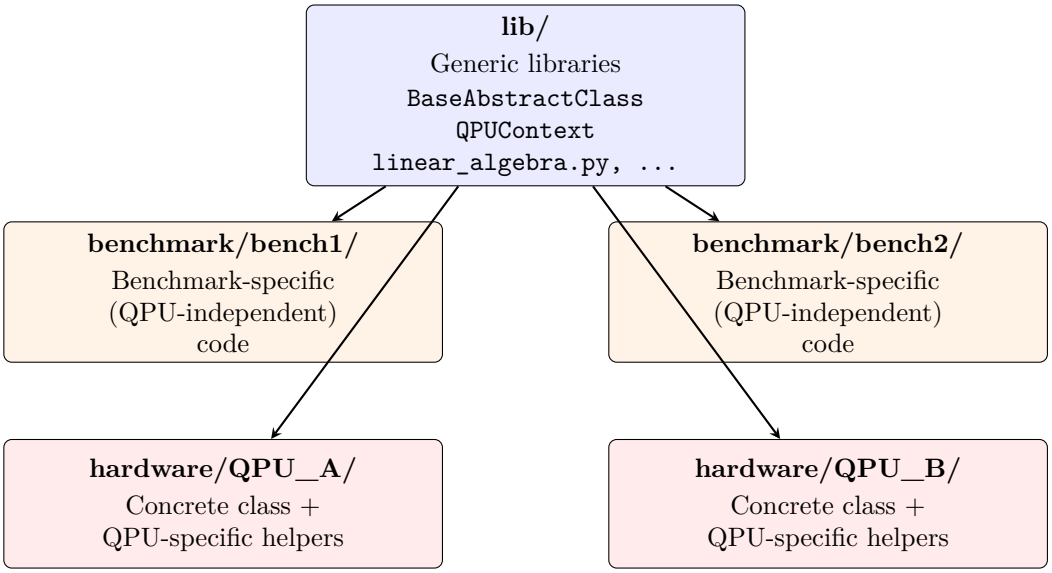
The preprocess, run and postprocess methods come from the dictionnary and is dynamically selected depending on the QPU backend and the target Benchmark.

1.6 Separation of Generic versus Specific Code

The code base is organised into three clearly distinct layers:

- **Generic libraries** (`lib/`) Contain all fully QPU-independent and benchmark-independent code, including the definition of the `BaseAbstractClass` (i.e. `QPUBenchClass`), the `QPUContext` data class, and reusable computational kernels.

- **Benchmark-specific code** (`benchmark/`) Lives under per-benchmark sub-directories (e.g. `benchmark/gscore`, `benchmark/linscore`, ...). These directories hold code that is specific to a given benchmark but remains independent of any particular QPU. They must implement the computation of the given benchmark score.
- **QPU-dependent code** (`hardware/`) Lives under per-QPU sub-directories (e.g. `hardware/QPUA`, `hardware/QPUB`, ...). These directories contain the concrete QPU classes together with all low-level helpers that talk to a specific hardware target.



- All algorithmic logic that does not depend on any particular QPU or benchmark belongs in the `lib/` directory.
- Code that is specific to a benchmark but remains independent of the underlying hardware resides under the corresponding `benchmark/<name>/` subdirectory. It is worth noting that there must be a QPU independent implementation of the benchmark that provides the computation to perform and the computation of the score.
- Only the code that realises a concrete QPU (including the three entries of `capabilitySet`) or that talks directly to a given QPU hardware target lives under the corresponding `hardware/QPUX/` subdirectory. These are the only files that are allowed to import QPU specific libraries (e.g. Qiskit, Ocean, *etc.*)

This strict separation maximises code reuse, keeps the generic framework stable, and makes the addition of a new application or a new QPU a local and well-contained change.

1.7 Summary

The combination of

- an abstract base class defining a uniform contract,
- a `capabilitySet` dictionary that always contains exactly three function pointers (`preprocess`, `run`, `postprocess`),
- a dedicated context object for data exchange, and
- a rigid four-step life-cycle (`__init__` → preprocess → run → postprocess)

gives the whole software a clean, extensible and easily maintainable architecture. Adding a new QPU reduces to writing one new concrete class that implements the abstract methods and registers the three capabilities; the rest of the framework remains untouched.

1.8 Short How-to: building a new support for a QPU

The following describe with more details than in section 1.4 the steps required to implement a new hardware/QPU support.

The global scheme of execution and the philosophy of implementation can be found on figure 1.1.

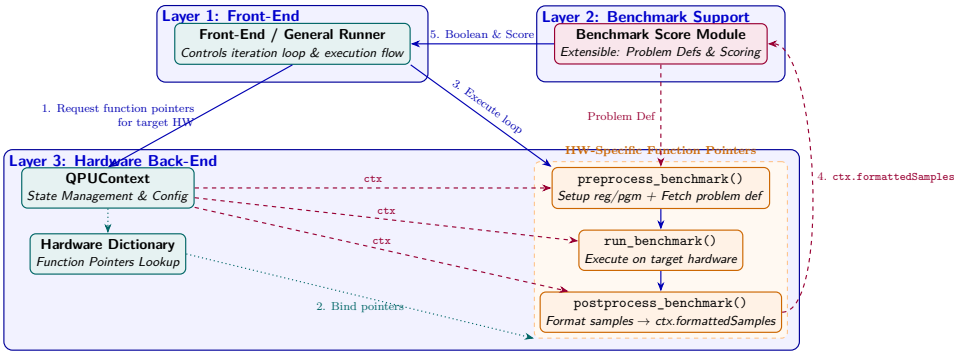


Figure 1.1: Three-layer decoupled architecture: The Front-End orchestrates execution, the pluggable Benchmark Layer defines problem sets and scoring, and the Hardware Back-End handles hardware-specific execution via dynamic function pointers.

1.8.1 Header

Adding support for a new hardware architecture requires creating a dedicated subdirectory inside `hardware/`. Best practice is to name this directory after the hardware architecture, keeping future variations and upgrades in mind.

Currently, the `hardware/` directory uses a flat structure. However, as support for more architectures grows, we may introduce an intermediate layer grouped by computing paradigm–such as Gate-based (e.g., superconducting, trapped ions, optical, etc.) or Quantum Simulators/"analog" (e.g., quantum annealers, neutral atoms, etc.).

For now, we will stick to the flat structure and simply create a single new subdirectory under hardware/. To set it up:

- Create a subdirectory named after the new hardware family.
- Add an `__init__.py` file (even if empty) so Python recognizes it as a package.
- Create at least one Python module to implement support for the architecture.

The following section outlines the key features and principles required to build a minimal Python support file.

```
import numpy as np
import [other generic support libraries for your code]
# add project root to python path
from pathlib import Path
root = Path(__file__).resolve().parents[2]
sys.path.insert(0, str(root))
# then we can import libraries from project
from lib.QPUBenchClass import *
from lib.linear_algebra_ising import *
# last but not least: import the relevant python
    libraries for the new QPU
import ... # e.g. qiskit, dimod, myqlm, etc.
```

Listing 1.3: Example of Python header for a QPU module

1.8.2 Backend functions

Next, you need to implement the core support functions. For each benchmark score, the module must provide the following three functions:

- `preprocess_benchmark`: Sets up the hardware registers and prepares the program to run the benchmark. Crucially, this function must utilize the generic problem definitions provided by the corresponding benchmark score modules.

- `run_benchmark`: Executes the benchmark. While execution logic is often generic across benchmarks on a given QPU, specific hardware constraints may require custom behavior.

- `postprocess_benchmark`: Processes and formats the QPU-specific samples collected in `QPUContext` during execution. It returns two items:

    1. A ternary flag indicating execution status (`True` to continue iterating, `False/0` when iterations are complete and the score can be computed). The ternary value (-1) is used if the iteration is required to continue without incrementing the counter value.
    2. A normalized sample format ready for processing by the generic benchmark score implementation.

All three functions share state primarily via the QPU context, receiving an `QPUContext` object as a parameter.

Naturally, developpers are free to implement any support function support they need to for a sound software design.

1.8.3 Concrete class implementation

Lastly, the module must provide a concrete class instanciation of the generic QPUBenchClass from whom it must inherit.

The important parts for data are:

- `capabilitySet`: provides the list of the benchmark scores supported by the QPU as a list of strings.

- `cpuDispatch`: is the main dictionnary that allows the main program to perform the callback mechanism. Each entry of the dictionnary have its key as the name of the benchmark score, and a list of 3 functions (callback functions) for preprocessing, running an iteration and postprocessing the iteration, as described in the previous section.

Regarding the class methods, the important elements to implement are:

- The constructor (`__init__`) that will be called by the main program to start the concrete instanciation. N.B.: the constructor must call the `selectBenchmark` method (see below).

- The `add_arguments` that provides the QPU-dependent arguments for the Command Line Interface (CLI) of the main program.

- The `selectBenchmark` method called by the constructor sets the `preprocessIteration`, `computeIteration` and `postprocessIteration` with the pointers to the backend functions that are in the dictionnary. Therefore, a call to

any of these 3 methods from the main program will utilize the correct backend function.

- The 3 class-backend methods `preprocessIteration`, `computeIteration` and `postprocessIteration` that must called the corresponding backend functions set by the `selectBenchmark` method.

This enables to provide a hardware and benchmark agnostic code execution, where the frontend will call seamlessly the correct and corresponding backend functions.

1.8.4 Hardware-Independent Sample Representation

The `postprocess_benchmark` function is responsible for populating the hardware-independent `ctx.formattedSample` attribute. The generic benchmark module relies on this structure to evaluate whether to stop iterating and to compute the final benchmark score. The formatted results must be structured as a tuple containing:

- The candidate solution vectors.
- The number of occurrences for each candidate solution.
- the third can be the associated cost/energy.

Based on this standardized format, the frontend benchmark function evaluates the stopping condition. Once iterations are complete, it computes the final score for the selected benchmark.

Appendices

1.A BACQ Benchmark

Implements the framework of the BACQ Benchmark suite in a plateform independent way. At the moment, it provides supports for the following QPUs:

- Generic CPU (for reference)
- Dwave
- (TBA) Pasqal (Ruby machine from Genci)

And the following scores:

- G-Score (Evaluation of Quantum Optimization heuristics)
- Lin2Score (Evaluation of capability for linear system solving in binary)

1.A.1 Prerequisites

- Python >= 3.10
- CMake >= 3.18
- Compilateur C++ (GCC/Clang)
- Doxygen
- Compilateur Latex (pdflatex)
- Pandoc

1.A.2 Installation

- Build & install

```
git clone [https://github.com/utilisateur/mon-projet.git
    ](https://github.com/utilisateur/mon-projet.git)
cd BacqBench
mkdir build && cd build
cmake ..
make
```

- Installation as python package (necessary for use with QPUs)
  - inside the main directory “cd ..” if required
  - activate your current QPU environement (e.g. DWave Ocean, Pasqal Pulser)
  - install benchmark package using “pip install -e .”
  - N.B. you can do it after runing the tests (see below) Installing in the support environement of your QPU is necessary to run the QPU backend of the benchmark suite.

1.A.3 Basic unitary tests

```
make check
```

This provides several unitary tests. Not all are implemented yet.

1.A.4 Usage

The main python script is BACQBenchmark at the root of the project The minimum parameter is the target QPU (or CPU comparison) E.g.

```
python3 BACQBenchmak dwave
```

Will launch the G-Score evaluation on the default architecture of DWave (Advantage/Pegasus ATM). It is worth noting that this requires the BACQBenchmak to be installed as package in the QPU environement. If it is not the case, the main program would display “Backend XXX not available” E.g.

```
python3 BACQBenchmak cpu
```

To run the G-Score on Simulated Annealing (CPU backend)
Use

```
python3 BACQBenchmak -h
```

or

```
python3 BACQBenchmak --help
```

To be provided with usage. Each backend should provide its own help, but in this case it is necessary to provide the backend on the command line
Being Alpha release, all options may not be implemented yet

1.A.5 To do

Here is a preliminary of pending functionalities:

- Saving log in json
- Evaluation of benchmark from json results file

1.B Doxygen documentation

BACQBench

Generated by Doxygen 1.9.8

1 Namespace Index 1
1.1 Namespace List . . . 1

2 Hierarchical Index 3
2.1 Class Hierarchy . . . 3

3 Class Index 5
3.1 Class List . . . 5

4 File Index 7
4.1 File List . . . 7

5 Namespace Documentation 9
5.1 BACQBenchmark Namespace Reference . . . 9
5.1.1 Detailed Description . . . 9
5.1.2 Function Documentation . . . 9
5.1.2.1 main() . . . 9
5.1.3 Variable Documentation . . . 10
5.1.3.1 backendClass . . . 10
5.1.3.2 backendMap . . . 10
5.1.3.3 BACKENDS . . . 10
5.1.3.4 modpath . . . 10
5.1.3.5 module . . . 10
5.2 benchmarks Namespace Reference . . . 11
5.3 benchmarks.linscore Namespace Reference . . . 11
5.4 benchmarks.linscore.LinBench2 Namespace Reference . . . 11
5.4.1 Function Documentation . . . 12
5.4.1.1 evalLin2Score() . . . 12
5.4.1.2 find2bitSolutions() . . . 12
5.4.1.3 fitSigmoid() . . . 12
5.4.1.4 generateAB() . . . 13
5.4.1.5 generateXrandom() . . . 13
5.4.1.6 Lin2Prob() . . . 13
5.4.1.7 LinProb() . . . 13
5.4.1.8 main() . . . 14
5.4.1.9 QUBOLin2Bench() . . . 14
5.4.1.10 successProbaLin2() . . . 14
5.4.1.11 truncate2bits() . . . 14
5.4.1.12 valcost() . . . 15
5.4.2 Variable Documentation . . . 15
5.4.2.1 root . . . 15
5.5 DWaveBenchmark Namespace Reference . . . 15
5.5.1 Detailed Description . . . 15
5.6 GnBenchLib Namespace Reference . . . 15

5.6.1 Detailed Description . . . 17
5.6.2 Function Documentation . . . 17
5.6.2.1 brokenConstraints() . . . 17
5.6.2.2 edgestosummitGn() . . . 17
5.6.2.3 evalGScore() . . . 17
5.6.2.4 GnMISadjacency() . . . 18
5.6.2.5 GnQUBO() . . . 18
5.6.2.6 gnScore() . . . 18
5.6.2.7 hammingDist() . . . 19
5.6.2.8 lowestBrokenConstraints() . . . 19
5.6.2.9 lowestCost() . . . 19
5.6.2.10 lowestHammingDist() . . . 19
5.6.2.11 maxValH() . . . 20
5.6.2.12 scoreRedux() . . . 20
5.6.2.13 simpleQUBO() . . . 20
5.6.2.14 simplification_score() . . . 20
5.6.2.15 Parameters . . . 20
5.6.2.16 Returns . . . 21
5.6.2.17 successProbaGn() . . . 21
5.6.3 Variable Documentation . . . 21
5.6.3.1 A . . . 21
5.6.3.2 A2_2d . . . 21
5.6.3.3 A2_3d . . . 22
5.6.3.4 end . . . 23
5.6.3.5 h . . . 23
5.6.3.6 J . . . 23
5.6.3.7 max . . . 23
5.6.3.8 myl . . . 23
5.6.3.9 myQ . . . 23
5.6.3.10 n . . . 23
5.6.3.11 N . . . 23
5.6.3.12 numval . . . 23
5.6.3.13 pseudosample . . . 24
5.6.3.14 redux2 . . . 24
5.6.3.15 redux3 . . . 24
5.6.3.16 root . . . 24
5.6.3.17 simplsingh . . . 24
5.6.3.18 simplsingJ . . . 24
5.6.3.19 simpQ . . . 24
5.6.3.20 tbc . . . 24
5.6.3.21 val . . . 24
5.7 hardware Namespace Reference . . . 24

5.8 hardware.cpu Namespace Reference . . . 25
5.9 hardware.cpu.CPUBenchmark Namespace Reference . . . 25
5.9.1 Detailed Description . . . 26
5.9.2 Function Documentation . . . 26
5.9.2.1 dummyLin2cpu() . . . 26
5.9.2.2 factorize_all() . . . 26
5.9.2.3 formatSelectSample() . . . 27
5.9.2.4 get_lib_path() . . . 27
5.9.2.5 load_library() . . . 27
5.9.2.6 lowestSamples() . . . 28
5.9.2.7 postprocessLin2() . . . 28
5.9.2.8 postprocessPrime() . . . 28
5.9.2.9 preprocessLin2() . . . 28
5.9.2.10 preprocessPrime() . . . 28
5.9.2.11 runFactorizeCPU() . . . 29
5.9.2.12 runSAmt_py() . . . 29
5.9.2.13 selectCPUheuristic() . . . 29
5.9.2.14 setGnSARegister() . . . 29
5.9.3 Variable Documentation . . . 30
5.9.3.1 argtypes . . . 30
5.9.3.2 BINARY . . . 30
5.9.3.3 CPUAlgoTypes . . . 30
5.9.3.4 dtype . . . 30
5.9.3.5 factors . . . 30
5.9.3.6 flags . . . 30
5.9.3.7 float64 . . . 30
5.9.3.8 int32 . . . 30
5.9.3.9 lib . . . 30
5.9.3.10 lib_path . . . 31
5.9.3.11 restype . . . 31
5.9.3.12 root . . . 31
5.9.3.13 SPIN . . . 31
5.10 hardware.dwave Namespace Reference . . . 31
5.11 hardware.dwave.Dwave_machines Namespace Reference . . . 31
5.11.1 Variable Documentation . . . 31
5.11.1.1 Dwave_Machines_dict . . . 31
5.12 hardware.dwave.DWaveBenchmark Namespace Reference . . . 31
5.12.1 Function Documentation . . . 32
5.12.1.1 formatSelectDwSample() . . . 32
5.12.1.2 generate_embedding() . . . 32
5.12.1.3 runDwave() . . . 33
5.12.1.4 sampleSetDwave() . . . 33

5.12.1.5 selectDWaveQPU() . . . 33
5.12.1.6 setGnDwaveRegister() . . . 33
5.12.1.7 setLin2DwaveRegister() . . . 34
5.12.2 Variable Documentation . . . 34
5.12.2.1 DWaveArchTypes . . . 34
5.13 hardware.dwave.DwaveMultiplier Namespace Reference . . . 34
5.14 hardware.dwave.Run_multiplier Namespace Reference . . . 34
5.14.1 Function Documentation . . . 35
5.14.1.1 preDwaveMultiplier() . . . 35
5.14.1.2 returnDwavePrime() . . . 35
5.14.1.3 save_dwave_txt() . . . 35
5.14.1.4 useDwaveMultiplier() . . . 35
5.15 lib Namespace Reference . . . 36
5.16 lib.linear_algebra_ising Namespace Reference . . . 36
5.16.1 Function Documentation . . . 36
5.16.1.1 duplicateIsing() . . . 36
5.16.1.2 duplicateQUBO() . . . 37
5.16.1.3 getHforDimod() . . . 37
5.16.1.4 getHQUBOforDimod() . . . 37
5.16.1.5 getJforDimod() . . . 37
5.16.1.6 IsingOutput() . . . 37
5.16.1.7 IsingtoQUBO() . . . 37
5.16.1.8 maxValIsing() . . . 37
5.16.1.9 maxValQUBO() . . . 37
5.16.1.10 QUBOfy() . . . 38
5.16.1.11 QUBOoutput() . . . 38
5.16.1.12 QUBOtoIsing() . . . 38
5.16.2 Variable Documentation . . . 38
5.16.2.1 A . . . 38
5.16.2.2 B . . . 38
5.16.2.3 ILp . . . 38
5.16.2.4 Iproblem . . . 38
5.16.2.5 numbit . . . 38
5.16.2.6 Q2 . . . 39
5.16.2.7 Qg . . . 39
5.16.2.8 QLp . . . 39
5.16.2.9 Qproblem . . . 39
5.16.2.10 sig . . . 39
5.17 lib.mylinearequations Namespace Reference . . . 39
5.17.1 Function Documentation . . . 39
5.17.1.1 eq2_2_s() . . . 39
5.17.1.2 eq2_2_u() . . . 40

5.17.1.3 eq2_3_s() . . . 40
5.17.1.4 eq2_3_u() . . . 40
5.17.1.5 eq3_2_2_1_u() . . . 40
5.17.1.6 eq3_2_u() . . . 40
5.17.1.7 eq3_3_u() . . . 40
5.17.1.8 g_1_qubo() . . . 41
5.17.1.9 g_1_simple_qubo() . . . 41
5.18 lib.QPUBenchClass Namespace Reference . . . 41
5.19 LinBench2 Namespace Reference . . . 41
5.19.1 Detailed Description . . . 41
5.20 prime-factorization Namespace Reference . . . 41
5.20.1 Function Documentation . . . 42
5.20.1.1 bestConfig() . . . 42
5.20.1.2 defMval() . . . 42
5.20.1.3 exportQUBOdict() . . . 42
5.20.1.4 HobofyPrime() . . . 42
5.20.1.5 listupdate() . . . 42
5.20.1.6 quadratize() . . . 43
5.20.1.7 reducePolynomial() . . . 43
5.20.2 Variable Documentation . . . 43
5.20.2.1 finalList2 . . . 43
5.20.2.2 list34 . . . 43
5.20.2.3 myh . . . 43
5.20.2.4 myJ . . . 43
5.20.2.5 mylist2 . . . 43
5.20.2.6 mylist3 . . . 43
5.20.2.7 mylist4 . . . 43
5.20.2.8 newlist2 . . . 44
5.20.2.9 tofactorize . . . 44
5.20.2.10 totvar . . . 44
5.20.2.11 valuedlist . . . 44
5.21 QPUBenchClass Namespace Reference . . . 44
5.21.1 Detailed Description . . . 44

6 Class Documentation **45**

6.1 hardware.cpu.CPUBenchmark.CPUBenchmark Class Reference . . . 45
6.1.1 Detailed Description . . . 47
6.1.2 Constructor & Destructor Documentation . . . 47
6.1.2.1 __init__() . . . 47
6.1.3 Member Function Documentation . . . 47
6.1.3.1 add_arguments() . . . 47
6.1.3.2 capabilities() . . . 48

6.1.3.3 compute() . . . 48
6.1.3.4 computeIteration() . . . 48
6.1.3.5 is_available() . . . 48
6.1.3.6 postprocess() . . . 48
6.1.3.7 postprocessIteration() . . . 48
6.1.3.8 preprocess() . . . 49
6.1.3.9 preprocessIteration() . . . 49
6.1.3.10 selectBenchmark() . . . 49
6.1.4 Member Data Documentation . . . 49
6.1.4.1 capabilitySet [1/2] . . . 49
6.1.4.2 capabilitySet [2/2] . . . 49
6.1.4.3 computeRes . . . 50
6.1.4.4 cpuctx . . . 50
6.1.4.5 cpuDispatch . . . 50
6.1.4.6 cpusAlgo . . . 50
6.1.4.7 currentBench . . . 50
6.1.4.8 dat . . . 50
6.1.4.9 isInit . . . 50
6.1.4.10 loginfile . . . 50
6.1.4.11 myCompute . . . 50
6.1.4.12 myPostprocess . . . 50
6.1.4.13 myPreprocess . . . 51
6.1.4.14 nIter [1/2] . . . 51
6.1.4.15 nIter [2/2] . . . 51
6.1.4.16 nqubits [1/2] . . . 51
6.1.4.17 nqubits [2/2] . . . 51
6.1.4.18 postprocessRes . . . 51
6.1.4.19 preprocessRes . . . 51
6.1.4.20 preprocessRes2 . . . 51
6.2 hardware.dwave.DWaveBenchmark.DWaveBenchmark Class Reference . . . 52
6.2.1 Detailed Description . . . 53
6.2.2 Constructor & Destructor Documentation . . . 53
6.2.2.1 __init__() . . . 53
6.2.3 Member Function Documentation . . . 54
6.2.3.1 add_arguments() . . . 54
6.2.3.2 capabilities() . . . 54
6.2.3.3 compute() . . . 54
6.2.3.4 computeIteration() . . . 54
6.2.3.5 is_available() . . . 54
6.2.3.6 postprocess() . . . 55
6.2.3.7 postprocessIteration() . . . 55
6.2.3.8 preprocess() . . . 55

6.2.3.9 preprocessIteration() . . . 55
6.2.3.10 selectBenchmark() . . . 55
6.2.4 Member Data Documentation . . . 55
6.2.4.1 capabilitySet [1/2] . . . 55
6.2.4.2 capabilitySet [2/2] . . . 56
6.2.4.3 computeRes . . . 56
6.2.4.4 currentBench . . . 56
6.2.4.5 dat . . . 56
6.2.4.6 dwarch . . . 56
6.2.4.7 dwaveDispatch . . . 56
6.2.4.8 dwctx . . . 56
6.2.4.9 dwembedding . . . 56
6.2.4.10 dwsampler . . . 56
6.2.4.11 isInit . . . 57
6.2.4.12 loginfile . . . 57
6.2.4.13 myCompute . . . 57
6.2.4.14 myPostprocess . . . 57
6.2.4.15 myPreprocess . . . 57
6.2.4.16 nIter [1/2] . . . 57
6.2.4.17 nIter [2/2] . . . 57
6.2.4.18 nqubits [1/2] . . . 57
6.2.4.19 nqubits [2/2] . . . 57
6.2.4.20 postprocessRes . . . 57
6.2.4.21 preprocessRes . . . 58
6.3 IsingModel Class Reference . . . 58
6.3.1 Constructor & Destructor Documentation . . . 58
6.3.1.1 IsingModel() . . . 58
6.3.2 Member Function Documentation . . . 59
6.3.2.1 discardCurrent() . . . 59
6.3.2.2 flip() . . . 59
6.3.2.3 generateCandidateConfiguration() . . . 59
6.3.2.4 getCurrentConfig() . . . 59
6.3.2.5 getCurrentEnergy() . . . 59
6.3.2.6 getEnergy() . . . 59
6.3.2.7 getPreviousEnergy() . . . 59
6.3.2.8 initModel() . . . 59
6.3.2.9 initRandomConfig() . . . 60
6.3.2.10 selectCurrent() . . . 60
6.3.3 Member Data Documentation . . . 60
6.3.3.1 currentConfig . . . 60
6.3.3.2 currentEnergy . . . 60
6.3.3.3 defaultModelVal . . . 60

6.3.3.4 isingSize . . . 60
6.3.3.5 isingType . . . 60
6.3.3.6 myh . . . 60
6.3.3.7 myJ . . . 60
6.3.3.8 previousConfig . . . 61
6.3.3.9 previousEnergy . . . 61
6.4 hardware.dwave.DwaveMultiplier.Multiplier Class Reference . . . 61
6.4.1 Constructor & Destructor Documentation . . . 62
6.4.1.1 __init__() . . . 62
6.4.2 Member Function Documentation . . . 62
6.4.2.1 build_graph_complete() . . . 62
6.4.2.2 cfa_nodes() . . . 62
6.4.2.3 cfa_tile_to_graph() . . . 62
6.4.2.4 Compatibility_w_Real_Graph() . . . 63
6.4.2.5 make_nodes() . . . 63
6.4.2.6 make_subgraph() . . . 63
6.4.2.7 plot_multiplier() . . . 63
6.4.2.8 read_data() . . . 64
6.4.3 Member Data Documentation . . . 64
6.4.3.1 biases . . . 64
6.4.3.2 binary_N . . . 64
6.4.3.3 cfa_plmt . . . 64
6.4.3.4 chain_strength . . . 64
6.4.3.5 Compatibility . . . 64
6.4.3.6 coords . . . 65
6.4.3.7 couplings . . . 65
6.4.3.8 Dwave_Machine . . . 65
6.4.3.9 factors_nodes . . . 65
6.4.3.10 H_range . . . 65
6.4.3.11 inputs_to_fix . . . 65
6.4.3.12 J_range . . . 65
6.4.3.13 N . . . 65
6.4.3.14 n . . . 65
6.4.3.15 n_col . . . 65
6.4.3.16 n_row . . . 66
6.4.3.17 offset . . . 66
6.4.3.18 pos . . . 66
6.4.3.19 sizes . . . 66
6.4.3.20 tile_0 . . . 66
6.4.3.21 topologie_graph . . . 66
6.5 hardware.dwave.Run_multiplier.Multiplier_to_Run Class Reference . . . 66
6.5.1 Constructor & Destructor Documentation . . . 67

6.5.1.1 __init__() . . . 67
6.5.2 Member Data Documentation . . . 67
6.5.2.1 Empirical_strength . . . 67
6.6 lib.QPUBenchClass.QPUBenchClass Class Reference . . . 68
6.6.1 Member Function Documentation . . . 68
6.6.1.1 add_arguments() . . . 68
6.6.1.2 capabilities() . . . 69
6.6.1.3 compute() . . . 69
6.6.1.4 is_available() . . . 69
6.6.1.5 postprocess() . . . 69
6.6.1.6 preprocess() . . . 69
6.6.1.7 selectBenchmark() . . . 69
6.7 lib.QPUBenchClass.QPUBenchClassCapability Class Reference . . . 70
6.7.1 Member Data Documentation . . . 70
6.7.1.1 name . . . 70
6.7.1.2 supports . . . 70
6.8 lib.QPUBenchClass.QPUBenchContext Class Reference . . . 71
6.8.1 Member Data Documentation . . . 71
6.8.1.1 arch . . . 71
6.8.1.2 computeRes . . . 71
6.8.1.3 currentBench . . . 71
6.8.1.4 dat . . . 72
6.8.1.5 distVectors . . . 72
6.8.1.6 formatedSamples . . . 72
6.8.1.7 nIter . . . 72
6.8.1.8 proba . . . 72
6.8.1.9 probaDist . . . 72
6.8.1.10 subarch . . . 72
6.8.1.11 target . . . 72
6.9 lib.QPUBenchClass.sampledResults Class Reference . . . 73
6.9.1 Member Data Documentation . . . 73
6.9.1.1 energyCost . . . 73
6.9.1.2 info . . . 73
6.9.1.3 num . . . 73
6.9.1.4 numQubits . . . 73
6.9.1.5 samples . . . 74
6.9.1.6 totNum . . . 74
6.10 lib.QPUBenchClass.samplesetClass Class Reference . . . 74
6.10.1 Member Data Documentation . . . 74
6.10.1.1 energy . . . 74
6.10.1.2 occurences . . . 74
6.10.1.3 vecVal . . . 74

6.11 SimulatedAnnealing Class Reference . . . 75
6.11.1 Constructor & Destructor Documentation . . . 75
6.11.1.1 SimulatedAnnealing() . . . 75
6.11.2 Member Function Documentation . . . 76
6.11.2.1 getBestEnergy() . . . 76
6.11.2.2 getCurrentConfig() . . . 76
6.11.2.3 initCoupling() . . . 76
6.11.2.4 result() . . . 76
6.11.2.5 run() . . . 76
6.11.2.6 setCurrentAsBest() . . . 76
6.11.2.7 setInitialTemperature() . . . 76
6.11.2.8 setScale() . . . 76
6.11.2.9 setSwipeNumber() . . . 77
6.11.3 Member Data Documentation . . . 77
6.11.3.1 bestEnergy . . . 77
6.11.3.2 configbuf . . . 77
6.11.3.3 deltaTemperature . . . 77
6.11.3.4 endingTemperature . . . 77
6.11.3.5 initTemperature . . . 77
6.11.3.6 lsize . . . 77
6.11.3.7 myIsing . . . 77
6.11.3.8 numSwipe . . . 77
6.11.3.9 scale . . . 77

7 File Documentation **79**

7.1 BACQBenchmark.py File Reference . . . 79
7.2 benchmarks/gscore/GnBenchLib.py File Reference . . . 79
7.3 benchmarks/linscore/LinBench2.py File Reference . . . 81
7.4 benchmarks/primescore/prime-factorization.py File Reference . . . 82
7.5 benchmarks/__init__.py File Reference . . . 82
7.6 benchmarks/linscore/__init__.py File Reference . . . 82
7.7 hardware/__init__.py File Reference . . . 82
7.8 hardware/cpu/__init__.py File Reference . . . 82
7.9 hardware/dwave/__init__.py File Reference . . . 82
7.10 lib/__init__.py File Reference . . . 82
7.11 hardware/cpu/CPUBenchmark.py File Reference . . . 82
7.12 hardware/dwave/Dwave_machines.py File Reference . . . 84
7.13 hardware/dwave/DWaveBenchmark.py File Reference . . . 84
7.14 hardware/dwave/DwaveMultiplier.py File Reference . . . 85
7.15 hardware/dwave/Run_multiplier.py File Reference . . . 85
7.16 hardware/src/factorizeOnCPU.cpp File Reference . . . 86
7.16.1 Detailed Description . . . 86

7.16.2 Typedef Documentation . . . 87
7.16.2.1 ll . . . 87
7.16.3 Function Documentation . . . 87
7.16.3.1 factorize_all() . . . 87
7.16.3.2 factorize_all_c() . . . 87
7.16.3.3 free_factorization() . . . 87
7.17 hardware/src/simpleSimulatedAnnealing.cpp File Reference . . . 87
7.17.1 Detailed Description . . . 88
7.17.2 Macro Definition Documentation . . . 88
7.17.2.1 OMP_PARALLEL . . . 88
7.17.3 Enumeration Type Documentation . . . 89
7.17.3.1 ModelType . . . 89
7.17.4 Function Documentation . . . 89
7.17.4.1 uniform() . . . 89
7.17.4.2 uniformDist() . . . 89
7.18 lib/linear_algebra_ising.py File Reference . . . 89
7.19 lib/mylinearequations.py File Reference . . . 90
7.20 lib/QPUBenchClass.py File Reference . . . 90

Index **91**

Chapter 1

Namespace Index

1.1 Namespace List

Here is a list of all namespaces with brief descriptions:

BACQBenchmark . . . 9
benchmarks . . . 11
benchmarks.linscore . . . 11
benchmarks.linscore.LinBench2 . . . 11
DWaveBenchmark
    This is the implementation of function sets for implementing DWave QPU particular access (clean Ocean SDK installed) . . . 15
GnBenchLib
    GnBenchLib is the generic (QPU independent) function sets for implementing the GScore . . . 15
hardware . . . 24
hardware.cpu . . . 25
hardware.cpu.CPUBenchmark . . . 25
hardware.dwave . . . 31
hardware.dwave.Dwave_machines . . . 31
hardware.dwave.DWaveBenchmark . . . 31
hardware.dwave.DwaveMultiplier . . . 34
hardware.dwave.Run_multiplier . . . 34
lib . . . 36
lib.linear_algebra_ising . . . 36
lib.mylinearequations . . . 39
lib.QPUBenchClass . . . 41
LinBench2
    LinBench2 is the implementation of function sets for implementing Lin2Bench and LinNBench benchmarks . . . 41
prime-factorization . . . 41
QPUBenchClass
    QPUBenchClass is the set of reference classes for project BACQ refering to management of multiple benchmark scores and multiple targets . . . 44

Chapter 2

Hierarchical Index

2.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

IsingModel 58
hardware.dwave.DwaveMultiplier.Multiplier 61
hardware.dwave.Run_multiplier.Multiplier_to_Run 66
lib.QPUBenchClass.QPUBenchClassCapability 70
lib.QPUBenchClass.QPUBenchContext 71
lib.QPUBenchClass.sampledResults 73
lib.QPUBenchClass.samplesetClass 74
SimulatedAnnealing 75
ABC
    lib.QPUBenchClass.QPUBenchClass 68
        hardware.cpu.CPUBenchmark.CPUBenchmark 45
        hardware.dwave.DWaveBenchmark.DWaveBenchmark 52

Generated by Doxygen

Chapter 3

Class Index

3.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

hardware.cpu.CPUBenchmark.CPUBenchmark 45
hardware.dwave.DWaveBenchmark.DWaveBenchmark 52
IsingModel 58
hardware.dwave.DwaveMultiplier.Multiplier 61
hardware.dwave.Run_multiplier.Multiplier_to_Run 66
lib.QPUBenchClass.QPUBenchClass 68
lib.QPUBenchClass.QPUBenchClassCapability 70
lib.QPUBenchClass.QPUBenchContext 71
lib.QPUBenchClass.sampledResults 73
lib.QPUBenchClass.samplesetClass 74
SimulatedAnnealing 75

Chapter 4

File Index

4.1 File List

Here is a list of all files with brief descriptions:

BACQBenchmark.py . . . 79
benchmarks/__init__.py . . . 82
benchmarks/gscore/GnBenchLib.py . . . 79
benchmarks/linscore/__init__.py . . . 82
benchmarks/linscore/LinBench2.py . . . 81
benchmarks/primescore/prime-factorization.py . . . 82
hardware/__init__.py . . . 82
hardware/cpu/__init__.py . . . 82
hardware/cpu/CPUBenchmark.py . . . 82
hardware/dwave/__init__.py . . . 82
hardware/dwave/Dwave_machines.py . . . 84
hardware/dwave/DWaveBenchmark.py . . . 84
hardware/dwave/DwaveMultiplier.py . . . 85
hardware/dwave/Run_multiplier.py . . . 85
hardware/src/factorizeOnCPU.cpp
Prime factorization using linear sieve algorithm Provides the implementation of function sets for implementing CPU/QPU comparison benchmarks for prime factorization using the linear sieve algorithm.
Dependencies are: . . . 86
hardware/src/simpleSimulatedAnnealing.cpp
Optimization heuristics implemented in C++ especially for CPU/QPU comparison Provides the implementation of function sets for implementing CPU/QPU comparison benchmarks for optimization heuristics. It provides both a usual binary background and a Ising implementation as well.
Dependencies are: . . . 87
lib/__init__.py . . . 82
lib/linear_algebra_ising.py . . . 89
lib/mylinearequations.py . . . 90
lib/QPUBenchClass.py . . . 90

Generated by Doxygen

Chapter 5

Namespace Documentation

5.1 BACQBenchmark Namespace Reference

Functions

- main ()

  Main implementation of BACQ Benchmark

Variables

- dict backendMap
- dict BACKENDS = {}

  BACKENDS is a list of available backends.
- str modpath = f"hardware.{backendName}.{moduleName}"

  try and import relevant backend library
- module = importlib.import_module(modpath, package=__name__)
- backendClass = getattr(module, className)

  get class within module

5.1.1 Detailed Description

```
Main stub for running BACQ benchmarking suite.
Allows multiple benchmarks and multiple backends (QPUs)
```

5.1.2 Function Documentation

5.1.2.1 main()

```
BACQBenchmark.main ( )
```

Main implementation of BACQ Benchmark

- Stub for every possible backend

Generated by Doxygen

Detailed implementation:

- Parser main (global and common) options
- create subparsers for possible backends
- option parsing depending on the requested backend
- then the bakckend adds its own arguments
- parsing arguments from CLI
- Dynamic instantiation of the selected backendClass
- For that use the class corresponding to the user's choice
- Remove general configuration flags to send only backend specific options to selected backend
- Magic instantiation through CLI arguments
- Backend execution (generic and use specific implementation through class instantiation)

5.1.3 Variable Documentation

5.1.3.1 backendClass

```
BACQBenchmark.backendClass = getattr(module, className)
```

get class within module

5.1.3.2 backendMap

```
dict BACQBenchmark.backendMap
```

Initial value:

```
00001 =  {"dwave": ("DWaveBenchmark", "DWaveBenchmark"),
00002     "cpu": ("CPUBenchmark", "CPUBenchmark")
00003     }
```

5.1.3.3 BACKENDS

```
dict BACQBenchmark.BACKENDS = {}
```

BACKENDS is a list of available backends.

5.1.3.4 modpath

```
str BACQBenchmark.modpath = f"hardware.{backendName}.{moduleName}"
```

try and import relevant backend library

5.1.3.5 module

```
BACQBenchmark.module = importlib.import_module(modpath, package=__name__)
```

5.2 benchmarks Namespace Reference

Namespaces

- namespace linscore

5.3 benchmarks.linscore Namespace Reference

Namespaces

- namespace LinBench2

5.4 benchmarks.linscore.LinBench2 Namespace Reference

Functions

- generateAB (Xmat, dyn=5, sign=1)

  Generate matrix A and vector B for Ax+B=0 equation from the planned solution Xmat.
- find2bitSolutions (Qmat)

  Find 2's complement solution.
- truncate2bits (Amat, Bmat)

  Generate binary matrices (2's complement encoding) from matrix A and vector B.
- generateXrandom (myr, dyn=20)

  Generate a random solution vector of size myr and range in dyn inputs: myr (int, size of vector solution), dyn (int, range for random number generation)
- valcost (vecX, numbit, h, J)

  Gives the minimum cost (optimal target) for the QUBO problem associated with AX+C=0.
- LinProb (myrange, bitnum)

  LinProb is the frontend function to implement the Lin∗Bench benchmark LinProb provides a random X vector solution and the problem instance that corresponds A matrix and B vector so that AX+B=0.
- Lin2Prob (myrange)

  Lin2Prob is the frontend function to implement the Lin2Bench benchmark It provides the A matrix, B vector and X solution vector with $X \in \{0,1\}^n$.
- QUBOLin2Bench (myrange)

  QUBOLin2Bench is one of the backend function to implement the Lin2Bench benchmark It gives a QUBO formulation of an instance of the problem to solve.
- successProbaLin2 (ctx, mysample)

  sucessProbaLin2 implements the evaluation of the success probability on one given instance of a problem it will rely on the already available optimal score computed in the previous function (QUBOLin2Bench)
- fitSigmoid (ctx, filename=None)

  fitSigmoid= fit a sigmoide function with the set of values $f(x) = \frac{1}{1+e^{-\lambda(x-x_0)}}$ *At the end the relevant value will be* x_0 *with its evaluated standard deviation But we also return the \lambda value with its standard deviation*
- evalLin2Score (ctx)

  evalLin2Score function implements the Lin2Bench score evaluation in an incremental fashion To be set as post-processing method according to QPU call and management
- main ()

  Main program: unitary and integration tests (when not used as a library)

Variables

- root = Path(__file__).resolve().parents[2]

5.4.1 Function Documentation

5.4.1.1 evalLin2Score()

```
benchmarks.linscore.LinBench2.evalLin2Score (
              ctx )
```

evalLin2Score function implements the Lin2Bench score evaluation in an incremental fashion To be set as post-processing method according to QPU call and management

- input: ctx is the QPU general context after computation

Detailed implementation:

- retrieve lowest energy samples from context
- update probability
- loop until we have at least 10 sets of samples
- Otherwise, set the condition for next iteration of the Lin2Bench
- test for the end of the benchmark

5.4.1.2 find2bitSolutions()

```
benchmarks.linscore.LinBench2.find2bitSolutions (
              Qmat )
```

Find 2's complement solution.

5.4.1.3 fitSigmoid()

```
benchmarks.linscore.LinBench2.fitSigmoid (
              ctx,
              filename = None )
```

fitSigmoid= fit a sigmoide function with the set of values $f(x) = \frac{1}{1+e^{-\lambda(x-x_0)}}$ At the end the relevant value will be x_0 with its evaluated standard deviation But we also return the \lambda value with its standard deviation

- input: QPU context with the distribution
  - optional input: file name to plot a graph (defaut None and no output graph)
  - output: returns estimated lambda with its standard deviation and x_0 with its standard deviation

Detailed implementation:

1. Get experimental results from context
2. Use sigmoid reciprocal function to fit a line, usign a change of variable: Y = ln(1/y - 1)
3. Propagate evaluated error on y according to the change of variable (limited development) : sigma↩ _Y = sigma_y / (y ∗ (1 - y))

Detailed implementation:

1. Linear regression with propagated uncertainties
2. Extract parameters of the sigmoid (\lambda and x0)
3. Set covariance matrix values to calculate standard deviation on x0
4. Calculate variance of x0

5.4.1.4 generateAB()

```
benchmarks.linscore.LinBench2.generateAB (
              Xmat,
              dyn = 5,
              sign = 1 )
```

Generate matrix A and vector B for Ax+B=0 equation from the planned solution Xmat.

- input Xmat: numpy vector target solution of the equation (in the benchmark we start from the solution)
- input dyn: integer is the range of random values for the

5.4.1.5 generateXrandom()

```
benchmarks.linscore.LinBench2.generateXrandom (
              myr,
              dyn = 20 )
```

Generate a random solution vector of size myr and range in dyn inputs: myr (int, size of vector solution), dyn (int, range for random number generation)

5.4.1.6 Lin2Prob()

```
benchmarks.linscore.LinBench2.Lin2Prob (
              myrange )
```

Lin2Prob is the frontend function to implement the Lin2Bench benchmark It provides the A matrix, B vector and X solution vector with $X \in \{0,1\}^n$.

- myrange is the current number of variables in the iterations.
- returns A matrix, and B and X vectors

5.4.1.7 LinProb()

```
benchmarks.linscore.LinBench2.LinProb (
              myrange,
              bitnum )
```

LinProb is the frontend function to implement the Lin$*$Bench benchmark LinProb provides a random X vector solution and the problem instance that corresponds A matrix and B vector so that AX+B=0.

- myrange is the current number of variables in the iterations
- bitnum is the current target resolution of the unknowns in number of bits
- returns A matrix, and B and X vectors

Detailed implementation:

1. Step 1: Compute bit mask for the solution vector X so that the solution is below 2^{n-1}
2. Step 2: generate random solution with the given bitmask
3. Step 3: generate A matrix and B vector so that AX+B=0
4. Final: returns the A matrix, B vector and X vector solution

5.4.1.8 main()

```
benchmarks.linscore.LinBench2.main ( )
```

Main program: unitary and integration tests (when not used as a library)

5.4.1.9 QUBOLin2Bench()

```
benchmarks.linscore.LinBench2.QUBOLin2Bench (
            myrange )
```

QUBOLin2Bench is one of the backend function to implement the Lin2Bench benchmark It gives a QUBO formulation of an instance of the problem to solve.

- myrange is the current number of variables in the iterations.
- returns QUBO matrix in h & J form, and associated minimal cost value

Detailed implementation:

- First: get the problem instance
- Second: compute QUBO formulation and minimum cost of the cost function
- return J matrix, h vector (dimod style) and calculated cost

5.4.1.10 successProbaLin2()

```
benchmarks.linscore.LinBench2.successProbaLin2 (
            ctx,
            mysample )
```

sucessProbaLin2 implements the evaluation of the success probability on one given instance of a problem it will rely on the already available optimal score computed in the previous function (QUBOLin2Bench)

- input: ctx (current context) and sample (the current set of selected samples)
- output: evaluated probability (note that there will be a need for statistical analysis of the results)

5.4.1.11 truncate2bits()

```
benchmarks.linscore.LinBench2.truncate2bits (
            Amat,
            Bmat )
```

Generate binary matrices (2's complement encoding) from matrix A and vector B.

5.4.1.12 valcost()

```
benchmarks.linscore.LinBench2.valcost (
              vecX,
              numbit,
              h,
              J )
```

Gives the minimum cost (optimal target) for the QUBO problem associated with AX+C=0.

- input: solution vector vecX (numpy vector), number of bits (resolution, int), h vector (numpy vector) and J (numpy matrix)

5.4.2 Variable Documentation

5.4.2.1 root

```
benchmarks.linscore.LinBench2.root = Path(__file__).resolve().parents[2]
```

5.5 DWaveBenchmark Namespace Reference

This is the implementation of function sets for implementing DWave QPU particular access (clean Ocean SDK installed).

5.5.1 Detailed Description

This is the implementation of function sets for implementing DWave QPU particular access (clean Ocean SDK installed).

Dependencies are:

- numpy
- linear algebra for Ising module/library
- QPUBenchClass library
- GnBenchLib library
- LinBench library
- Dwave libraries (Ocean Toolkit) Distributed under EU Public Licence (EUPL) 1.2 and (c) CEA LIST, QuLIST team Initial release 0.1 (not feature complete) 2026Q4

5.6 GnBenchLib Namespace Reference

GnBenchLib is the generic (QPU independent) function sets for implementing the GScore.

Functions

- maxValH (h)

  Returns extremal value within a given vector.
- GnQUBO (n, Verbose=False)

  Generate Gn QUBO matrix.
- edgestosummitGn (n)

  Compute the corresponding sets between Gn nodes and Gn edges Input: value of n for Gn Output: double vector for corresponding set between edges and nodes.
- simpleQUBO (E)

  generate simplified QUBO matrix with values in {-1,0,1} input: numpy matrix for QUBO output: simplified QUBO matrix (numpy)
- GnMISadjacency (n)

  define the adjacency matrix for MIS solving
  input: n value for Gn output: adjacency matrix for the MIS formulation of Gn
- hammingDist (n, sample)

  Compute the Haming distance from optimal solution of Gn series in standard numbering of edges to the actual found solution Input: value of n for Gn , vector candidate solution Output: integer= Hamming distance= number of discrepencies in the bit string of the provided vector.
- lowestHammingDist (n, sample)

  Search the set of samples with the lowest Haming distance to the optimal.
- brokenConstraints (n, sample)

  Compute the number of broken constraints in the candidate vector solution.
- lowestBrokenConstraints (n, sample)

  Search the lowest set of samples with the lowest number of broken constraints.
- gnScore (nfailed, proba, sample, redux=0.0)

  Compute GScore from.
- successProbaGn (n, sampledResults samples)

  Return an estimator of the success probability for the Gn optimization outcome.
- lowestCost (n, samples)

  Search minimal cost from samples for Gn.
- evalGScore (ctx)
- (float, int) simplification_score (np.ndarray A, np.ndarray A_prime, float lambda_pen=1)

  Evaluate graph simplification in terms of equivalent number of nodes.
- float scoreRedux (int n, np.ndarray A, np.ndarray Ap)

Variables

- root = Path(__file__).resolve().parents[2]
- n = int(sys.argv[1])
- myQ
- numval
- N = len(myQ)
- h
- J
- end
- simpQ = simpleQUBO(myQ)
- simpIsingh
- simpIsingJ
- max = maxValH(h)
- int val = 0
- myl = len(h)
- bool tbc = False

- A = GnMISadjacency(n)
- A2_3d
- A2_2d
- float redux2 = scoreRedux(2, A, A2_2d)
- pseudosample = sampledResults(np.array([[1,1,1,1,1,1,1,1,1,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0]]),1,1,1E6,27,None)
- float redux3 = scoreRedux(2, A, A2_3d)

5.6.1 Detailed Description

GnBenchLib is the generic (QPU independent) function sets for implementing the GScore.

Dependencies are:

- numpy
- linear algebra for Ising module/library
- QPUBenchClass library

5.6.2 Function Documentation

5.6.2.1 brokenConstraints()

```
GnBenchLib.brokenConstraints (
              n,
              sample )
```

Compute the number of broken constraints in the candidate vector solution.

- Input: integer value of n from Gn, binary vector candidate solution
- Ouput: integer, number of broken constraints
- prerequisite: the numbering of nodes is the traditional one; the (n+1)∗∗2 first variables are the optimal solution

5.6.2.2 edgestosummitGn()

```
GnBenchLib.edgestosummitGn (
              n )
```

Compute the corresponding sets between Gn nodes and Gn edges Input: value of n for Gn Output: double vector for corresponding set between edges and nodes.

5.6.2.3 evalGScore()

```
GnBenchLib.evalGScore (
              ctx )
```

5.6.2.4 GnMISadjacency()

```
GnBenchLib.GnMISadjacency (
              n )
```

define the adjacency matrix for MIS solving
input: n value for Gn output: adjacency matrix for the MIS formulation of Gn

5.6.2.5 GnQUBO()

```
GnBenchLib.GnQUBO (
              n,
              Verbose = False )
```

Generate Gn QUBO matrix.

- Input: value of n (integer, positive) for the Gn
- Output: Numpy QUBO matrix for the Gn using the value n provided as input

Detailed implementation:

1. First pass= numbering the nodes of the Gn graph
2. Second pass= on edges between u and v nodes
3. Finally= generate QUBO matrix from graph

5.6.2.6 gnScore()

```
GnBenchLib.gnScore (
              nfailed,
              proba,
              sample,
              redux = 0.0 )
```

Compute GScore from.

- input 1: the failed n rank (nfailed),
- input 2: the probability of finding the correct answer at nfailed-1
- input 3: the best set of samples
- input 4 (optional): for approximate embedding, the reduction score to be applied (default 0 = none)
- output: returns the evaluation of the G-Score

5.6.2.7 hammingDist()

```
GnBenchLib.hammingDist (
              n,
              sample )
```

Compute the Haming distance from optimal solution of Gn series in standard numbering of edges to the actual found solution Input: value of n for Gn , vector candidate solution Output: integer= Hamming distance= number of discrepencies in the bit string of the provided vector.

5.6.2.8 lowestBrokenConstraints()

```
GnBenchLib.lowestBrokenConstraints (
              n,
              sample )
```

Search the lowest set of samples with the lowest number of broken constraints.

- input 1: n value of G_n
- input 2: sample set of samples
- output: lowest number of broken constraints in the sample set.

5.6.2.9 lowestCost()

```
GnBenchLib.lowestCost (
              n,
              samples )
```

Search minimal cost from samples for Gn.

- input: n value for Gn, and set of total samples
- return: set of samples with the minimal QUBO cost

5.6.2.10 lowestHammingDist()

```
GnBenchLib.lowestHammingDist (
              n,
              sample )
```

Search the set of samples with the lowest Haming distance to the optimal.

- input 1: n value of G_n
- input 2: sample or set of samples
- output: lowest Haming distance in the sample/sample set.

5.6.2.11 maxValH()

```
GnBenchLib.maxValH (
              h )
```

Returns extremal value within a given vector.

- input: vector h
- output: value from maximum absolute value found

5.6.2.12 scoreRedux()

```
float GnBenchLib.scoreRedux (
             int n,
             np.ndarray A,
             np.ndarray Ap )
```

5.6.2.13 simpleQUBO()

```
GnBenchLib.simpleQUBO (
              E )
```

generate simplified QUBO matrix with values in {-1,0,1} input: numpy matrix for QUBO output: simplified QUBO matrix (numpy)

5.6.2.14 simplification_score()

```
(float, int) GnBenchLib.simplification_score (
             np.ndarray A,
             np.ndarray A_prime,
             float  lambda_pen = 1 )
```

Evaluate graph simplification in terms of equivalent number of nodes.

The returned score lies in [alpha, n], where:

- n = number of nodes in the original graph
- alpha = cardinality of a maximum independent set (or a lower bound)

5.6.2.15 Parameters

A : np.ndarray Adjacency matrix of the original graph (n x n) A_prime : np.ndarray Adjacency matrix of the transformed graph (same size with zero rows for pruned nodes, or smaller) lambda_pen : float Penalty factor applied to degree increases (default = 1. as maybe for future extension)

5.6.2.16 Returns

tuple of boolean and float:

- float= score reductio ratio in the simplified graph wrt the initial graph
- int= total number of edges in the intial graph = boolean

5.6.2.17 successProbaGn()

```
GnBenchLib.successProbaGn (
              n,
             sampledResults samples )
```

Return an estimator of the success probability for the Gn optimization outcome.

- Inputs : n value of Gn, and samples
- Output: estimated probability (takes into account insufficient statistical relevance)

5.6.3 Variable Documentation

5.6.3.1 A

```
GnBenchLib.A = GnMISadjacency(n)
```

5.6.3.2 A2_2d

```
GnBenchLib.A2_2d
```

Initial value:

```
00001 =  np.array([
00002         [ 0, 0, 0, 0, 0, 0, 0, 0, 0,  0, 1, 1, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0 ], #0
      #0
00003         [ 0, 0, 0, 0, 0, 0, 0, 0, 0,  0, 0, 1, 0, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0 ], #1
      #0
00004         [ 0, 0, 0, 0, 0, 0, 0, 0, 0,  0, 0, 0, 0, 0, 0, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0 ], #2
      #0
00005         [ 0, 0, 0, 0, 0, 0, 0, 0, 0,  0, 0, 0, 1, 1, 0, 1, 0, 0, 1, 0, 1, 0, 1, 0, 0, 0 ], #3
      #0
00006         [ 0, 0, 0, 0, 0, 0, 0, 0, 0,  0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0 ], #4
      #4
00007         [ 0, 0, 0, 0, 0, 0, 0, 0, 0,  0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0 ], #5
      #4
00008         [ 0, 0, 0, 0, 0, 0, 0, 0, 0,  0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 1, 0 ], #6
      #0
00009         [ 0, 0, 0, 0, 0, 0, 0, 0, 0,  0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 1, 1 ], #7
      #0
00010         [ 0, 0, 0, 0, 0, 0, 0, 0, 0,  0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 1, 0, 0 ], #8
      #0
00011         [ 0, 0, 0, 0, 0, 0, 0, 0, 0,  0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0 ], #9
      #6
00012         [ 1, 0, 0, 0, 0, 0, 0, 0, 0,  0, 0, 1, 0, 0, 1, 0, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0 ], #10
      #1
00013         [ 1, 1, 0, 0, 0, 0, 0, 0, 0,  0, 1, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0 ], #11
      #2
00014         [ 0, 0, 0, 1, 0, 0, 0, 0, 0,  0, 0, 0, 0, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0 ], #12
      #1
00015         [ 0, 1, 0, 1, 1, 0, 0, 0, 0,  0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0 ], #13
      #1
```

```
00016         [ 0, 1, 0, 0, 0, 0, 0, 0, 0,  0, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0 ], #14
      #1
00017         [ 0, 0, 1, 1, 0, 1, 0, 0, 0,  0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0 ], #15
      #1
00018         [ 0, 0, 1, 0, 0, 0, 0, 0, 0,  0, 1, 0, 1, 0, 0, 1, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0 ], #16
      #1
00019         [ 1, 0, 1, 0, 0, 0, 0, 0, 0,  0, 1, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0 ], #17
      #2
00020         [ 0, 0, 0, 1, 0, 0, 0, 0, 0,  0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 1, 1, 0, 1, 0 ], #18
      #1
00021         [ 0, 0, 0, 0, 0, 0, 0, 0, 0,  0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0 ], #19
      #6
00022         [ 0, 0, 0, 1, 0, 1, 0, 0, 1,  0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 1, 0, 0, 0 ], #20
      #1
00023         [ 0, 0, 0, 0, 0, 0, 1, 0, 0,  0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1 ], #21
      #1
00024         [ 0, 0, 0, 1, 1, 0, 0, 1, 0,  0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 1, 0 ], #22
      #1
00025         [ 0, 0, 0, 0, 0, 0, 0, 0, 1,  0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 1, 1, 0, 0, 0, 0, 1 ], #23
      #1
00026         [ 0, 0, 0, 0, 0, 0, 1, 1, 0,  0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 1, 0 ], #24
      #2
00027         [ 0, 0, 0, 0, 0, 0, 0, 1, 0,  0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 1, 1, 0, 1, 0, 0 ], #25
      #1
00028         [ 0, 0, 0, 0, 0, 0, 1, 0, 1,  0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 1, 0, 0, 0 ], #26
      #2
00029         ])
```

5.6.3.3 A2_3d

GnBenchLib.A2_3d

Initial value:

```
00001 =  np.array([
00002         [ 0, 0, 0, 0, 0, 0, 0, 0, 0,  0, 1, 1, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0 ], #0
      #0
00003         [ 0, 0, 0, 0, 0, 0, 0, 0, 0,  0, 0, 1, 0, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0 ], #1
      #0
00004         [ 0, 0, 0, 0, 0, 0, 0, 0, 0,  0, 0, 0, 0, 0, 0, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0 ], #2
      #0
00005         [ 0, 0, 0, 0, 0, 0, 0, 0, 0,  0, 0, 0, 1, 1, 0, 1, 0, 0, 1, 0, 1, 0, 1, 0, 0, 0, 0 ], #3
      #0
00006         [ 0, 0, 0, 0, 0, 0, 0, 0, 0,  0, 0, 0, 1, 1, 0, 0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0, 0 ], #4
      #2
00007         [ 0, 0, 0, 0, 0, 0, 0, 0, 0,  0, 0, 0, 1, 0, 0, 1, 0, 0, 1, 0, 1, 0, 0, 0, 0, 0, 0 ], #5
      #2
00008         [ 0, 0, 0, 0, 0, 0, 0, 0, 0,  0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 1, 0, 1 ], #6
      #0
00009         [ 0, 0, 0, 0, 0, 0, 0, 0, 0,  0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 1, 1, 0 ], #7
      #0
00010         [ 0, 0, 0, 0, 0, 0, 0, 0, 0,  0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 1, 0, 0, 1 ], #8
      #0
00011         [ 1, 1, 1, 0, 0, 0, 0, 0, 0,  0, 1, 0, 1, 0, 1, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0 ], #9
      #1
00012         [ 1, 0, 0, 0, 0, 0, 0, 0, 0,  1, 0, 1, 0, 0, 1, 0, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0 ], #10
      #0
00013         [ 1, 1, 0, 0, 0, 0, 0, 0, 0,  0, 1, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0 ], #11
      #2
00014         [ 0, 0, 0, 1, 0, 0, 0, 0, 0,  1, 0, 0, 0, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0 ], #12
      #0
00015         [ 0, 1, 0, 1, 1, 0, 0, 0, 0,  0, 0, 0, 1, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0 ], #13
      #1
00016         [ 0, 1, 0, 0, 0, 0, 0, 0, 0,  1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0 ], #14
      #0
00017         [ 0, 0, 1, 1, 0, 1, 0, 0, 0,  0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0 ], #15
      #1
00018         [ 0, 0, 1, 0, 0, 0, 0, 0, 0,  1, 1, 0, 1, 0, 0, 1, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0 ], #16
      #0
00019         [ 1, 0, 1, 0, 0, 0, 0, 0, 0,  0, 1, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0 ], #17
      #2
00020         [ 0, 0, 0, 1, 0, 0, 0, 0, 0,  0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 0, 1, 1, 0, 1, 0 ], #18
      #0
00021         [ 0, 0, 0, 0, 0, 0, 1, 1, 1,  0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 1, 0, 1, 0, 1, 0 ], #19
      #1
00022         [ 0, 0, 0, 1, 0, 1, 0, 0, 1,  0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 1, 0, 0, 0 ], #20
      #1
00023         [ 0, 0, 0, 0, 0, 0, 1, 0, 0,  0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 1, 1, 1, 1 ], #21
      #0
00024         [ 0, 0, 0, 1, 1, 0, 0, 1, 0,  0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 1, 0 ], #22
      #1
00025         [ 0, 0, 0, 0, 0, 0, 0, 0, 1,  0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 0, 0, 0, 0, 1 ], #23
      #0
```

```
00026         [ 0, 0, 0, 0, 0, 0, 1, 1, 0,  0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 1, 0 ], #24
      #2
00027         [ 0, 0, 0, 0, 0, 0, 0, 1, 0,  0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 0, 1, 1, 0, 1, 0, 0 ], #25
      #0
00028         [ 0, 0, 0, 0, 0, 0, 1, 0, 1,  0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 1, 0, 0, 0 ], #26
      #2
00029         ])
```

5.6.3.4 end

```
GnBenchLib.end
```

5.6.3.5 h

```
GnBenchLib.h
```

5.6.3.6 J

```
GnBenchLib.J
```

5.6.3.7 max

```
GnBenchLib.max = maxValH(h)
```

5.6.3.8 myl

```
GnBenchLib.myl = len(h)
```

5.6.3.9 myQ

```
GnBenchLib.myQ
```

5.6.3.10 n

```
GnBenchLib.n = int(sys.argv[1])
```

5.6.3.11 N

```
GnBenchLib.N = len(myQ)
```

5.6.3.12 numval

```
GnBenchLib.numval
```

5.6.3.13 pseudosample

```
GnBenchLib.pseudosample = sampledResults(np.array([[1,1,1,1,1,1,1,1,1,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0]]),1
```

5.6.3.14 redux2

```
float GnBenchLib.redux2 = scoreRedux(2, A, A2_2d)
```

5.6.3.15 redux3

```
float GnBenchLib.redux3 = scoreRedux(2, A, A2_3d)
```

5.6.3.16 root

```
GnBenchLib.root = Path(__file__).resolve().parents[2]
```

5.6.3.17 simpIsingh

```
GnBenchLib.simpIsingh
```

5.6.3.18 simpIsingJ

```
GnBenchLib.simpIsingJ
```

5.6.3.19 simpQ

```
GnBenchLib.simpQ = simpleQUBO(myQ)
```

5.6.3.20 tbc

```
bool GnBenchLib.tbc = False
```

5.6.3.21 val

```
GnBenchLib.val = 0
```

5.7 hardware Namespace Reference

Namespaces

- namespace cpu
- namespace dwave

5.8 hardware.cpu Namespace Reference

Namespaces

- namespace CPUBenchmark

5.9 hardware.cpu.CPUBenchmark Namespace Reference

Classes

- class CPUBenchmark

Functions

- ctypes.CDLL load_library (str lib_name, Optional[Path] relative_path=None)

  Sideload a library in a multi-plateform way.
- Path get_lib_path (str lib_name)

  Return library path according to underlying OS.
- selectCPUheuristic (algoType='sa')

  Select base algorithm for CPU base benchmark.
- runSAmt_py (np.ndarray J, np.ndarray h, int mytype, int nsamp)

  calling C function runSAmt from Python3
- setGnSARegister (niter, logfile=None, Verbose=False)

  Preprocessing for Gn series: set J and h so that runSAmt can use them to solve Gn.
- lowestSamples (samples)
- formatSelectSample (sampleset, lowest=True)

  Transform CPU pseudo samples to sampledResults class input: sample set output: sampledResults samples.
- list[list[int]] factorize_all (int n, int time_limit_sec=60)

  Python bridge to the C++ factorize_all function.
- runFactorizeCPU (nmin, nmax, dum, samplenb)

  Use CPU function using a sieve algorithm to sample the benchmark
- preprocessPrime (n)

  dummy preprocess for prime factorization
- postprocessPrime (ctx)

  dummy postprocess for prime factorization.
- preprocessLin2 (n)
- dummyLin2cpu (nmin, nmax, dum, samplenb)
- postprocessLin2 (ctx)

  dummy postprocess for lin1score.

Generated by Doxygen

Variables

- root = Path(__file__).resolve().parents[2]
- Path lib_path = get_lib_path("BacqBench")
- lib = ctypes.CDLL(str(lib_path))
- dict CPUAlgoTypes = {'sa', 'sqa'}

  'sa' for simulated annealing, and 'sqa' for simulated quantum annealing (Monte-Carlo simulation)
- int BINARY = 0
- int SPIN = 1
- argtypes
- dtype
- float64
- flags
- int32
- restype
- list[list[int]] factors = factorize_all(20, time_limit_sec=30)

5.9.1 Detailed Description

```
@file CPUBenchmark.py
@package CPUBenchmark is the implementation of function sets for implementing CPU/QPU comparison benchmarks.
Dependencies are:
    - numpy
    - linear algebra for Ising module/library
    - QPUBenchClass library
    - GnBenchLib library
    - argparse python module
    Distributed under EU Public Licence (EUPL) and (c) CEA LIST, QuLIST team
```

5.9.2 Function Documentation

5.9.2.1 dummyLin2cpu()

```
hardware.cpu.CPUBenchmark.dummyLin2cpu (
              nmin,
              nmax,
              dum,
              samplenb )
```

5.9.2.2 factorize_all()

```
list[list[int]] hardware.cpu.CPUBenchmark.factorize_all (
             int n,
             int  time_limit_sec = 60 )
```

Python bridge to the C++ factorize_all function.

Returns a list of lists of ints.

5.9.2.3 formatSelectSample()

```
hardware.cpu.CPUBenchmark.formatSelectSample (
              sampleset,
              lowest = True )
```

Transform CPU pseudo samples to sampledResults class input: sample set output: sampledResults samples.

5.9.2.4 get_lib_path()

```
Path hardware.cpu.CPUBenchmark.get_lib_path (
             str lib_name )
```

Return library path according to underlying OS.

Detailed implementation:

default path is within current directory

- Case of Windows
- Case of MacOS
- Case of Linux/Unix Side load the C++ SA library to use within implementation of the computing with CPU implementation Sideload the library for SA (simpleSA) and prime factorization (factorize_all_↩ c) CPU Algorithms available for this implementation of reference score N.B.: sqa not yet implemented (Simulated Quantum Annealing)

5.9.2.5 load_library()

```
ctypes.CDLL hardware.cpu.CPUBenchmark.load_library (
             str lib_name,
             Optional[Path]  relative_path = None )
```

Sideload a library in a multi-plateform way.

- lib_name : base name (without extension, as the extension is defined by the OS)

Detailed implementation:

1. Define a path if not given

- Default behavior : same directory as the current file or in ./libs/

1. Base OS detection

- case of Windows
- Case of MacOS
- default Linux and other Unices

5.9.2.6 lowestSamples()

```
hardware.cpu.CPUBenchmark.lowestSamples (
              samples )
```

Detailed implementation:

find lowest energy Gather by vector values prepare output layout for the samples now, return as a list

5.9.2.7 postprocessLin2()

```
hardware.cpu.CPUBenchmark.postprocessLin2 (
              ctx )
```

dummy postprocess for lin1score.

We return 1E6 which is a very rough estimate of average CPU capability with usual libraries

Detailed implementation:

initialize the remaining as default "uninitialized"

5.9.2.8 postprocessPrime()

```
hardware.cpu.CPUBenchmark.postprocessPrime (
              ctx )
```

dummy postprocess for prime factorization.

We can imagine that the best Top 500 computers could achieve around 55 bits for continuous factorization in the interval [2,n] primescore=log_2 (n) within 24h

5.9.2.9 preprocessLin2()

```
hardware.cpu.CPUBenchmark.preprocessLin2 (
              n )
```

5.9.2.10 preprocessPrime()

```
hardware.cpu.CPUBenchmark.preprocessPrime (
              n )
```

dummy preprocess for prime factorization

5.9.2.11 runFactorizeCPU()

```
hardware.cpu.CPUBenchmark.runFactorizeCPU (
              nmin,
              nmax,
              dum,
              samplenb )
```

Use CPU function using a sieve algorithm to sample the benchmark

5.9.2.12 runSAmt_py()

```
hardware.cpu.CPUBenchmark.runSAmt_py (
             np.ndarray J,
             np.ndarray h,
             int mytype,
             int nsamp )
```

calling C function runSAmt from Python3

- input: matrix J (numpy), vector h (numpy), type of variables, number of samples
- returns: SA results (set of samples) Python3 caller for C++ function runSAmt

Detailed implementation:

enforce memory contiguity for J and h as C++ parameters safety checks (matching sizes) allocate return values on the python side calling the C function return samples

5.9.2.13 selectCPUheuristic()

```
hardware.cpu.CPUBenchmark.selectCPUheuristic (
              algoType = 'sa' )
```

Select base algorithm for CPU base benchmark.

The default is 'sa' (Simulated Annealing)

Detailed implementation:

import dependencies using ctypes for importing C/C++ functions signature is: double runSA(int msize, ModelType mtype, double ∗theJ, double ∗theh, int ∗ret) it is the same for the MT (openMP) version

5.9.2.14 setGnSARegister()

```
hardware.cpu.CPUBenchmark.setGnSARegister (
              niter,
              logfile = None,
              Verbose = False )
```

Preprocessing for Gn series: set J and h so that runSAmt can use them to solve Gn.

Detailed implementation:

extract J matrix and h vector from myQ (QUBO matrix of the Gn instance)

5.9.3 Variable Documentation

5.9.3.1 argtypes

```
hardware.cpu.CPUBenchmark.argtypes
```

5.9.3.2 BINARY

```
int hardware.cpu.CPUBenchmark.BINARY = 0
```

5.9.3.3 CPUAlgoTypes

```
dict hardware.cpu.CPUBenchmark.CPUAlgoTypes = {'sa', 'sqa'}
```

'sa' for simulated annealing, and 'sqa' for simulated quantum annealing (Monte-Carlo simulation)

5.9.3.4 dtype

```
hardware.cpu.CPUBenchmark.dtype
```

5.9.3.5 factors

```
list[list[int]] hardware.cpu.CPUBenchmark.factors = factorize_all(20, time_limit_sec=30)
```

5.9.3.6 flags

```
hardware.cpu.CPUBenchmark.flags
```

5.9.3.7 float64

```
hardware.cpu.CPUBenchmark.float64
```

5.9.3.8 int32

```
hardware.cpu.CPUBenchmark.int32
```

5.9.3.9 lib

```
hardware.cpu.CPUBenchmark.lib = ctypes.CDLL(str(lib_path))
```

5.9.3.10 lib_path

```
Path hardware.cpu.CPUBenchmark.lib_path = get_lib_path("BacqBench")
```

5.9.3.11 restype

```
hardware.cpu.CPUBenchmark.restype
```

5.9.3.12 root

```
hardware.cpu.CPUBenchmark.root = Path(__file__).resolve().parents[2]
```

5.9.3.13 SPIN

```
int hardware.cpu.CPUBenchmark.SPIN = 1
```

5.10 hardware.dwave Namespace Reference

Namespaces

- namespace Dwave_machines
- namespace DWaveBenchmark
- namespace DwaveMultiplier
- namespace Run_multiplier

5.11 hardware.dwave.Dwave_machines Namespace Reference

Variables

- dict Dwave_Machines_dict

5.11.1 Variable Documentation

5.11.1.1 Dwave_Machines_dict

```
dict hardware.dwave.Dwave_machines.Dwave_Machines_dict
```

5.12 hardware.dwave.DWaveBenchmark Namespace Reference

Classes

- class DWaveBenchmark

Functions

- selectDWaveQPU (archType='pegasus')

  Select DWave QPU by using architecture and requiring an available QPU with the given architecture.
- generate_embedding (BinaryQuadraticModel bqm, DWaveSampler sampler, list[int] seeds, bool return_↩ num_qubits=False)

  Generate embeddings for the bqm on the sample with the given seeds.
- BinaryQuadraticModel setGnDwaveRegister (mysampler, niter, ctx, logfile=None, Verbose=False)

  setGnDwaveRegister set the DWave register to solve the Gn problem
- BinaryQuadraticModel setLin2DwaveRegister (mysampler, niter, ctx, logfile=None, Verbose=False)

  setLin2DwaveRegister set the DWave register to solve the Lin2Bench problem
- runDwave (sampler=None, embedding=None, model=None)

  Use DWave procedures to solve the problem on the QPU You can refer to DWave ocean documentation The principle is to use the "better" fixed minor-embedding found by the use of generate_embedding function input: valid sampler class refering to an actual DWave QPU, fixed embedding, and a QUBO/Ising model to run on the QPU output: Dwave ocean sample class.
- sampleSetDwave (myresponse=None)
- formatSelectDwSample (dwsampleset, lowest=True)

  Transform DWave samples from Sample Ocean class to sampledResults class input: DW sample output: sampled↩ Results samples.

Variables

- dict DWaveArchTypes = {'pegasus', 'zephyr'}

  DWave Available Architectures= make it evolve through the commercial availability of QPUs.

5.12.1 Function Documentation

5.12.1.1 formatSelectDwSample()

```
hardware.dwave.DWaveBenchmark.formatSelectDwSample (
              dwsampleset,
              lowest = True )
```

Transform DWave samples from Sample Ocean class to sampledResults class input: DW sample output: sampled↩ Results samples.

Detailed implementation:

initialize the remaining as default "uninitialized"

5.12.1.2 generate_embedding()

```
hardware.dwave.DWaveBenchmark.generate_embedding (
             BinaryQuadraticModel bqm,
             DWaveSampler sampler,
             list[int] seeds,
             bool return_num_qubits = False )
```

Generate embeddings for the bqm on the sample with the given seeds.

Returns the one with the minimum number of qubits. If return_num_qubits=True it returns the best_embedding the number of qubits in the best embedding else returns the best embedding

Detailed implementation:

generate embeddings using a series of random seeds and the standard embedding heuristic from DWave find the one with the lowest number of qubits= lowest number of qubits usually correlates with better results select the best embedding wrt the number of qubits for the problem Count the number of qubits in this embedding

5.12.1.3 runDwave()

```
hardware.dwave.DWaveBenchmark.runDwave (
              sampler = None,
              embedding = None,
              model = None )
```

Use DWave procedures to solve the problem on the QPU You can refer to DWave ocean documentation The principle is to use the "better" fixed minor-embedding found by the use of generate_embedding function input: valid sampler class refering to an actual DWave QPU, fixed embedding, and a QUBO/Ising model to run on the QPU output: Dwave ocean sample class.

Be carreful it is not directly compatible with the formatedSamples class from the BACQ libraries

5.12.1.4 sampleSetDwave()

```
hardware.dwave.DWaveBenchmark.sampleSetDwave (
              myresponse = None )
```

5.12.1.5 selectDWaveQPU()

```
hardware.dwave.DWaveBenchmark.selectDWaveQPU (
              archType = 'pegasus' )
```

Select DWave QPU by using architecture and requiring an available QPU with the given architecture.

5.12.1.6 setGnDwaveRegister()

```
BinaryQuadraticModel hardware.dwave.DWaveBenchmark.setGnDwaveRegister (
              mysampler,
              niter,
              ctx,
              logfile = None,
              Verbose = False )
```

setGnDwaveRegister set the DWave register to solve the Gn problem

- Inputs are:
  - sampler (DWave QPU ID)
  - niter, iteration number
  - logfile (N.B.: not yet implemented TBA)
  - verbose flag (default False)
- Output: model (ready to run on QPU), associated minor-embedding, and number of qubits

Detailed implementation:

1. Prepare model by calling GnQUBO function from GnBenchLib
2. Generate minor embedding

5.12.1.7 setLin2DwaveRegister()

```
BinaryQuadraticModel hardware.dwave.DWaveBenchmark.setLin2DwaveRegister (
              mysampler,
              niter,
              ctx,
              logfile = None,
              Verbose = False )
```

setLin2DwaveRegister set the DWave register to solve the Lin2Bench problem

- Inputs are:
  - sampler (DWave QPU ID)
  - niter, iteration number
  - logfile (N.B.: not yet implemented TBA)
  - verbose flag (default False)
- Output: model (ready to run on QPU), associated minor-embedding, and number of qubits

Detailed implementation:

1. Prepare model by calling GnQUBO function from GnBenchLib
2. Generate minor embedding

5.12.2 Variable Documentation

5.12.2.1 DWaveArchTypes

```
dict hardware.dwave.DWaveBenchmark.DWaveArchTypes = {'pegasus', 'zephyr'}
```

DWave Available Architectures= make it evolve through the commercial availability of QPUs.

5.13 hardware.dwave.DwaveMultiplier Namespace Reference

Classes

- class Multiplier

5.14 hardware.dwave.Run_multiplier Namespace Reference

Classes

- class Multiplier_to_Run

Functions

- save_dwave_txt (sampleset, filepath, title="RESULTS")
- returnDwavePrime (ctx)
- preDwaveMultiplier (sampler, ni, ctx)
- useDwaveMultiplier (sampler, ni, ctx)

  Function to run the benchmark on specified integer Input:

5.14.1 Function Documentation

5.14.1.1 preDwaveMultiplier()

```
hardware.dwave.Run_multiplier.preDwaveMultiplier (
              sampler,
              ni,
              ctx )
```

5.14.1.2 returnDwavePrime()

```
hardware.dwave.Run_multiplier.returnDwavePrime (
              ctx )
```

5.14.1.3 save_dwave_txt()

```
hardware.dwave.Run_multiplier.save_dwave_txt (
              sampleset,
              filepath,
              title = "RESULTS" )
```

5.14.1.4 useDwaveMultiplier()

```
hardware.dwave.Run_multiplier.useDwaveMultiplier (
              sampler,
              ni,
              ctx )
```

Function to run the benchmark on specified integer Input:

- arch: either pegasus or zephyr
- toFactorize: list of number to factorize
- This function will hide several details from the Dwave implementation and are ATM quite dependent on the actual layout of the QPU (it targets very specific QPUs using their working qubits and working links)

Detailed implementation:

1. Run the Multiplier on the Dwave Machine 65

5.15 lib Namespace Reference

Namespaces

- namespace linear_algebra_ising
- namespace mylinearequations
- namespace QPUBenchClass

5.16 lib.linear_algebra_ising Namespace Reference

Functions

- QUBOfy (Amat, Bvect, bitsize, sig=1)

  transform Ax+B=0 linear system into a QUBO cost function
- QUBOtoIsing (myQUBO)
- IsingtoQUBO (myH, myJ)
- QUBOoutput (myQUBO)
- IsingOutput (myH, myJ)
- getHforDimod (myh)
- getHQUBOforDimod (myJ)
- getJforDimod (myJ)
- maxValQUBO (myQUBO)
- maxValIsing (myH, myJ)
- duplicateQUBO (myQUBO, num, cs, wantDiag=True, isLoop=True)
- duplicateIsing (myH, myJ, num, cs, isLoop=True)

Variables

- A
- B
- numbit
- sig
- Qg = np.array([[1,0],[0,-1]])
- Qproblem = QUBOfy(A,B,numbit,sig)
- Iproblem = QUBOtoIsing(Qproblem)
- Q2 = IsingtoQUBO(Iproblem[0],Iproblem[1])
- QLp = duplicateQUBO(Qproblem, 2, 0.0, True)
- ILp = duplicateIsing(Iproblem[0],Iproblem[1],3, 18)

5.16.1 Function Documentation

5.16.1.1 duplicateIsing()

```
lib.linear_algebra_ising.duplicateIsing (
              myH,
              myJ,
              num,
              cs,
              isLoop = True )
```

5.16.1.2 duplicateQUBO()

```
lib.linear_algebra_ising.duplicateQUBO (
              myQUBO,
              num,
              cs,
              wantDiag = True,
              isLoop = True )
```

5.16.1.3 getHforDimod()

```
lib.linear_algebra_ising.getHforDimod (
              myh )
```

5.16.1.4 getHQUBOforDimod()

```
lib.linear_algebra_ising.getHQUBOforDimod (
              myJ )
```

5.16.1.5 getJforDimod()

```
lib.linear_algebra_ising.getJforDimod (
              myJ )
```

5.16.1.6 IsingOutput()

```
lib.linear_algebra_ising.IsingOutput (
              myH,
              myJ )
```

5.16.1.7 IsingtoQUBO()

```
lib.linear_algebra_ising.IsingtoQUBO (
              myH,
              myJ )
```

5.16.1.8 maxValIsing()

```
lib.linear_algebra_ising.maxValIsing (
              myH,
              myJ )
```

5.16.1.9 maxValQUBO()

```
lib.linear_algebra_ising.maxValQUBO (
              myQUBO )
```

5.16.1.10 QUBOfy()

```
lib.linear_algebra_ising.QUBOfy (
              Amat,
              Bvect,
              bitsize,
              sig = 1 )
```

transform Ax+B=0 linear system into a QUBO cost function

5.16.1.11 QUBOoutput()

```
lib.linear_algebra_ising.QUBOoutput (
              myQUBO )
```

5.16.1.12 QUBOtoIsing()

```
lib.linear_algebra_ising.QUBOtoIsing (
              myQUBO )
```

5.16.2 Variable Documentation

5.16.2.1 A

```
lib.linear_algebra_ising.A
```

5.16.2.2 B

```
lib.linear_algebra_ising.B
```

5.16.2.3 ILp

```
lib.linear_algebra_ising.ILp = duplicateIsing(Iproblem[0],Iproblem[1],3, 18)
```

5.16.2.4 Iproblem

```
lib.linear_algebra_ising.Iproblem = QUBOtoIsing(Qproblem)
```

5.16.2.5 numbit

```
lib.linear_algebra_ising.numbit
```

5.16.2.6 Q2

```
lib.linear_algebra_ising.Q2 = IsingtoQUBO(Iproblem[0],Iproblem[1])
```

5.16.2.7 Qg

```
lib.linear_algebra_ising.Qg = np.array([[1,0],[0,-1]])
```

5.16.2.8 QLp

```
lib.linear_algebra_ising.QLp = duplicateQUBO(Qproblem, 2, 0.0, True)
```

5.16.2.9 Qproblem

```
lib.linear_algebra_ising.Qproblem = QUBOfy(A,B,numbit,sig)
```

5.16.2.10 sig

```
lib.linear_algebra_ising.sig
```

5.17 lib.mylinearequations Namespace Reference

Functions

- eq3_3_u (A, B, numbit, sig)
- eq2_2_u (A, B, numbit, sig)
- eq2_2_s (A, B, numbit, sig)
- eq2_3_u (A, B, numbit, sig)
- eq2_3_s (A, B, numbit, sig)
- eq3_2_u (A, B, numbit, sig)
- eq3_2_2_1_u (A, B, numbit, sig)
- g_1_qubo (Q)
- g_1_simple_qubo (Q)

5.17.1 Function Documentation

5.17.1.1 eq2_2_s()

```
lib.mylinearequations.eq2_2_s (
              A,
              B,
              numbit,
              sig )
```

5.17.1.2 eq2_2_u()

```
lib.mylinearequations.eq2_2_u (
              A,
              B,
              numbit,
              sig )
```

5.17.1.3 eq2_3_s()

```
lib.mylinearequations.eq2_3_s (
              A,
              B,
              numbit,
              sig )
```

5.17.1.4 eq2_3_u()

```
lib.mylinearequations.eq2_3_u (
              A,
              B,
              numbit,
              sig )
```

5.17.1.5 eq3_2_2_1_u()

```
lib.mylinearequations.eq3_2_2_1_u (
              A,
              B,
              numbit,
              sig )
```

5.17.1.6 eq3_2_u()

```
lib.mylinearequations.eq3_2_u (
              A,
              B,
              numbit,
              sig )
```

5.17.1.7 eq3_3_u()

```
lib.mylinearequations.eq3_3_u (
              A,
              B,
              numbit,
              sig )
```

5.17.1.8 g_1_qubo()

```
lib.mylinearequations.g_1_qubo (
              Q )
```

5.17.1.9 g_1_simple_qubo()

```
lib.mylinearequations.g_1_simple_qubo (
              Q )
```

5.18 lib.QPUBenchClass Namespace Reference

Classes

- class QPUBenchClass
- class QPUBenchClassCapability
- class QPUBenchContext
- class sampledResults
- class samplesetClass

5.19 LinBench2 Namespace Reference

LinBench2 is the implementation of function sets for implementing Lin2Bench and LinNBench benchmarks.

5.19.1 Detailed Description

LinBench2 is the implementation of function sets for implementing Lin2Bench and LinNBench benchmarks.

Dependencies are:

- numpy
- linear algebra for Ising module/library
- QPUBenchClass library

Distributed under EU Public Licence (EUPL) 1.2 and (c) CEA LIST, QuLIST team

5.20 prime-factorization Namespace Reference

Functions

- listupdate (mylist, myelement)
- HobofyPrime (N, nbits, pbits)
- bestConfig (numbit, hobo)
- quadratize (nvar, initlist)
- defMval (mylist, myM)
- reducePolynomial (numvar, myDinom)
- exportQUBOdict (mylist)

Variables

- int tofactorize = 15
- mylist2
- mylist3
- mylist4
- list34 = mylist3+mylist4
- totvar
- newlist2
- valuedlist = defMval(newlist2,200.0)
- finalList2 = reducePolynomial(totvar,mylist2+valuedlist)
- myh
- myJ

5.20.1 Function Documentation

5.20.1.1 bestConfig()

```
prime-factorization.bestConfig (
              numbit,
              hobo )
```

5.20.1.2 defMval()

```
prime-factorization.defMval (
              mylist,
              myM )
```

5.20.1.3 exportQUBOdict()

```
prime-factorization.exportQUBOdict (
              mylist )
```

5.20.1.4 HobofyPrime()

```
prime-factorization.HobofyPrime (
              N,
              nbits,
              pbits )
```

5.20.1.5 listupdate()

```
prime-factorization.listupdate (
              mylist,
              myelement )
```

5.20.1.6 quadratize()

```
prime-factorization.quadratize (
              nvar,
              initlist )
```

5.20.1.7 reducePolynomial()

```
prime-factorization.reducePolynomial (
              numvar,
              myDinom )
```

5.20.2 Variable Documentation

5.20.2.1 finalList2

```
prime-factorization.finalList2 = reducePolynomial(totvar,mylist2+valuedlist)
```

5.20.2.2 list34

```
prime-factorization.list34 = mylist3+mylist4
```

5.20.2.3 myh

```
prime-factorization.myh
```

5.20.2.4 myJ

```
prime-factorization.myJ
```

5.20.2.5 mylist2

```
prime-factorization.mylist2
```

5.20.2.6 mylist3

```
prime-factorization.mylist3
```

5.20.2.7 mylist4

```
prime-factorization.mylist4
```

5.20.2.8 newlist2

```
prime-factorization.newlist2
```

5.20.2.9 tofactorize

```
int prime-factorization.tofactorize = 15
```

5.20.2.10 totvar

```
prime-factorization.totvar
```

5.20.2.11 valuedlist

```
prime-factorization.valuedlist = defMval(newlist2,200.0)
```

5.21 QPUBenchClass Namespace Reference

QPUBenchClass is the set of reference classes for project BACQ refering to management of multiple benchmark scores and multiple targets.

5.21.1 Detailed Description

QPUBenchClass is the set of reference classes for project BACQ refering to management of multiple benchmark scores and multiple targets.

Dependencies are:

- ABC
- typing
- numpy
- argparse
- dataclass

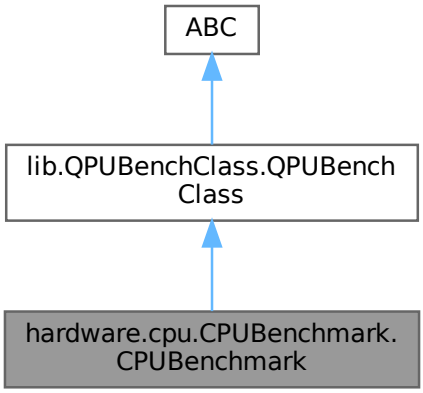
Distributed under EU Public Licence (EUPL) and (c) CEA LIST, QuLIST team

Chapter 6

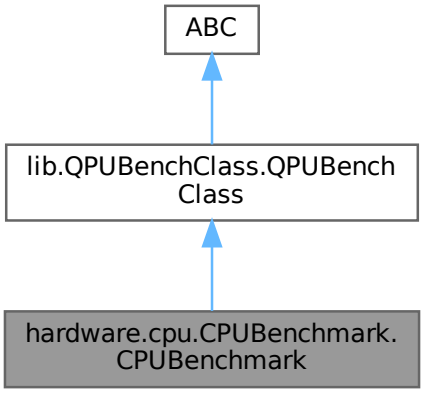
Class Documentation

6.1 hardware.cpu.CPUBenchmark.CPUBenchmark Class Reference

Inheritance diagram for hardware.cpu.CPUBenchmark.CPUBenchmark:



Collaboration diagram for hardware.cpu.CPUBenchmark.CPUBenchmark:



Public Member Functions

- QPUBenchClassCapability capabilities (self)

  Method to return capabilities of QPU benchmark implementations.
- add_arguments (cls, argparse.ArgumentParser parser)

  This class method provides plateform specific parsing option for command line interface.
- bool is_available (self, qpuArch=None)

  Return availability for QPU type.
- selectBenchmark (self, str bench, int num, cpualgo=None, ∗∗kwargs)

  Select Benchmark (QPU agnostic call, but concrete implementation)
- bool preprocessIteration (self, str bench, ∗∗kwargs)
- bool computeIteration (self, ∗∗kwargs)
- bool postprocessIteration (self, ∗∗kwargs)
- __init__ (self, iteration, bench, alg, logfile=None)
- preprocess (self)
- compute (self)
- postprocess (self)

Public Attributes

- capabilitySet
- nIter
- nqubits
- postprocessRes

Static Public Attributes

- list capabilitySet = ['qscore', 'gscore', 'lin2score', 'linvol', 'primescore']
  set of capabilities for CPUs
- cpuctx = QPUBenchContext()
- cpusAlgo = None
- currentBench = None
- myPreprocess = None
- myCompute = None
- myPostprocess = None
- int nIter = 0
- preprocessRes = None
- preprocessRes2 = None
- computeRes = None
- bool isInit = False
- loginfile = None
- dat = None
- int nqubits = -1
- dict cpuDispatch = {'gscore':[setGnSARegister, runSAmt_py, evalGScore], 'primescore':[preprocessPrime, runFactorizeCPU, postprocessPrime], 'lin2score':[preprocessLin2, dummyLin2cpu, postprocessLin2]}

6.1.1 Detailed Description

```
Concrete implementation of class QPUBenchClass for CPUs
```

6.1.2 Constructor & Destructor Documentation

6.1.2.1 __init__()

```
hardware.cpu.CPUBenchmark.CPUBenchmark.__init__ (
              self,
              iteration,
              bench,
              alg,
              logfile = None )
```

6.1.3 Member Function Documentation

6.1.3.1 add_arguments()

```
hardware.cpu.CPUBenchmark.CPUBenchmark.add_arguments (
             cls,
            argparse.ArgumentParser parser )
```

This class method provides plateform specific parsing option for command line interface.

- For CPU it will provide the base algorithm to select (atm, only SA)

Reimplemented from lib.QPUBenchClass.QPUBenchClass.

6.1.3.2 capabilities()

```
QPUBenchClassCapability hardware.cpu.CPUBenchmark.CPUBenchmark.capabilities (
              self )
```

Method to return capabilities of QPU benchmark implementations.

Reimplemented from lib.QPUBenchClass.QPUBenchClass.

6.1.3.3 compute()

```
hardware.cpu.CPUBenchmark.CPUBenchmark.compute (
              self )
```

Reimplemented from lib.QPUBenchClass.QPUBenchClass.

6.1.3.4 computeIteration()

```
bool hardware.cpu.CPUBenchmark.CPUBenchmark.computeIteration (
              self,
             ** kwargs )
```

6.1.3.5 is_available()

```
bool hardware.cpu.CPUBenchmark.CPUBenchmark.is_available (
              self,
              qpuArch = None )
```

Return availability for QPU type.

- for QPU this is always True

Reimplemented from lib.QPUBenchClass.QPUBenchClass.

6.1.3.6 postprocess()

```
hardware.cpu.CPUBenchmark.CPUBenchmark.postprocess (
              self )
```

Reimplemented from lib.QPUBenchClass.QPUBenchClass.

6.1.3.7 postprocessIteration()

```
bool hardware.cpu.CPUBenchmark.CPUBenchmark.postprocessIteration (
              self,
             ** kwargs )
```

6.1.3.8 preprocess()

```
hardware.cpu.CPUBenchmark.CPUBenchmark.preprocess (
              self )
```

Reimplemented from lib.QPUBenchClass.QPUBenchClass.

6.1.3.9 preprocessIteration()

```
bool hardware.cpu.CPUBenchmark.CPUBenchmark.preprocessIteration (
              self,
             str bench,
             ** kwargs )
```

Detailed implementation:

Preprocessing: also QPU agnostic, concrete implementation for CPU

6.1.3.10 selectBenchmark()

```
hardware.cpu.CPUBenchmark.CPUBenchmark.selectBenchmark (
              self,
             str bench,
             int num,
              cpualgo = None,
             ** kwargs )
```

Select Benchmark (QPU agnostic call, but concrete implementation)

Detailed implementation:

Example: select gscore at n=2

Reimplemented from lib.QPUBenchClass.QPUBenchClass.

6.1.4 Member Data Documentation

6.1.4.1 capabilitySet [1/2]

```
list hardware.cpu.CPUBenchmark.CPUBenchmark.capabilitySet = ['qscore', 'gscore', 'lin2score',
'linvol', 'primescore']  [static]
```

set of capabilities for CPUs

6.1.4.2 capabilitySet [2/2]

```
hardware.cpu.CPUBenchmark.CPUBenchmark.capabilitySet
```

6.1.4.3 computeRes

```
hardware.cpu.CPUBenchmark.CPUBenchmark.computeRes = None  [static]
```

6.1.4.4 cpuctx

```
hardware.cpu.CPUBenchmark.CPUBenchmark.cpuctx = QPUBenchContext()  [static]
```

6.1.4.5 cpuDispatch

```
dict hardware.cpu.CPUBenchmark.CPUBenchmark.cpuDispatch = {'gscore':[setGnSARegister, runSAmt_py,
evalGScore], 'primescore':[preprocessPrime, runFactorizeCPU, postprocessPrime], 'lin2score'↩
:[preprocessLin2, dummyLin2cpu, postprocessLin2]}  [static]
```

6.1.4.6 cpusAlgo

```
hardware.cpu.CPUBenchmark.CPUBenchmark.cpusAlgo = None  [static]
```

6.1.4.7 currentBench

```
hardware.cpu.CPUBenchmark.CPUBenchmark.currentBench = None  [static]
```

6.1.4.8 dat

```
hardware.cpu.CPUBenchmark.CPUBenchmark.dat = None  [static]
```

6.1.4.9 isInit

```
bool hardware.cpu.CPUBenchmark.CPUBenchmark.isInit = False  [static]
```

6.1.4.10 loginfile

```
hardware.cpu.CPUBenchmark.CPUBenchmark.loginfile = None  [static]
```

6.1.4.11 myCompute

```
hardware.cpu.CPUBenchmark.CPUBenchmark.myCompute = None  [static]
```

6.1.4.12 myPostprocess

```
hardware.cpu.CPUBenchmark.CPUBenchmark.myPostprocess = None  [static]
```

6.1.4.13 myPreprocess

```
hardware.cpu.CPUBenchmark.CPUBenchmark.myPreprocess = None  [static]
```

6.1.4.14 nIter [1/2]

```
int hardware.cpu.CPUBenchmark.CPUBenchmark.nIter = 0  [static]
```

6.1.4.15 nIter [2/2]

```
hardware.cpu.CPUBenchmark.CPUBenchmark.nIter
```

6.1.4.16 nqubits [1/2]

```
int hardware.cpu.CPUBenchmark.CPUBenchmark.nqubits = -1  [static]
```

6.1.4.17 nqubits [2/2]

```
hardware.cpu.CPUBenchmark.CPUBenchmark.nqubits
```

6.1.4.18 postprocessRes

```
hardware.cpu.CPUBenchmark.CPUBenchmark.postprocessRes
```

6.1.4.19 preprocessRes

```
hardware.cpu.CPUBenchmark.CPUBenchmark.preprocessRes = None  [static]
```

6.1.4.20 preprocessRes2

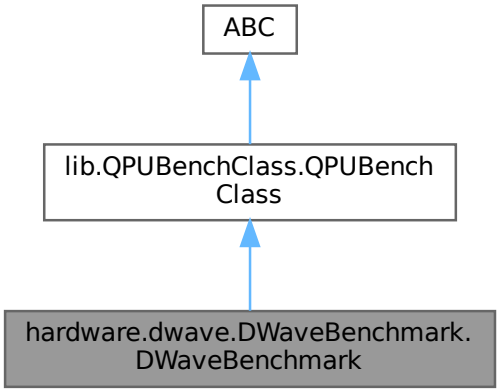
```
hardware.cpu.CPUBenchmark.CPUBenchmark.preprocessRes2 = None  [static]
```

The documentation for this class was generated from the following file:

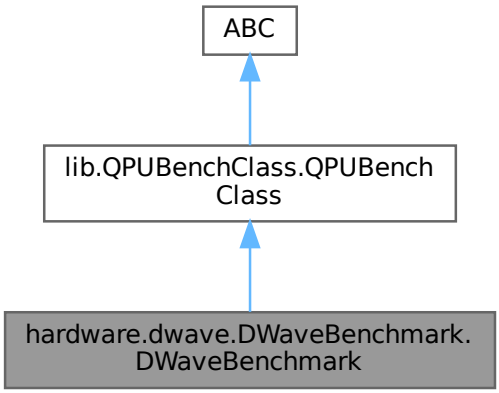
- hardware/cpu/CPUBenchmark.py

6.2 hardware.dwave.DWaveBenchmark.DWaveBenchmark Class Reference

Inheritance diagram for hardware.dwave.DWaveBenchmark.DWaveBenchmark:



Collaboration diagram for hardware.dwave.DWaveBenchmark.DWaveBenchmark:



Public Member Functions

- QPUBenchClassCapability capabilities (self)
- add_arguments (cls, argparse.ArgumentParser parser)

  Generic class method to add arguments to the cli invocation depending on instancied class.

- bool is_available (self, qpuArch=None)
- selectBenchmark (self, str bench, int num, qpuArch=None, ∗∗kwargs)
- bool preprocessIteration (self, str bench, ∗∗kwargs)
- bool computeIteration (self, ∗∗kwargs)
- bool postprocessIteration (self, ∗∗kwargs)
- __init__ (self, iteration, bench, arch, logfile=None)
- preprocess (self)
- compute (self)
- postprocess (self)

Public Attributes

- capabilitySet
- nIter
- nqubits
- postprocessRes

Static Public Attributes

- list capabilitySet = ['qscore', 'gscore', 'lin2score', 'linsolverany', 'primescore']
  set of capabilities for DWave QPUs
- dwctx = QPUBenchContext()
- dwsampler = None
- dwembedding = None
- dwarch = None
- currentBench = None
- myPreprocess = None
- myCompute = None
- myPostprocess = None
- int nIter = 0
- preprocessRes = None
- computeRes = None
- bool isInit = False
- loginfile = None
- dat = None
- int nqubits = -1
- dict dwaveDispatch

6.2.1 Detailed Description

```
Concrete implementation of class QPUBenchClass for DWave family of QPUs
```

6.2.2 Constructor & Destructor Documentation

6.2.2.1 __init__()

```
hardware.dwave.DWaveBenchmark.DWaveBenchmark.__init__ (
            self,
            iteration,
            bench,
            arch,
            logfile = None )
```

6.2.3 Member Function Documentation

6.2.3.1 add_arguments()

```
hardware.dwave.DWaveBenchmark.DWaveBenchmark.add_arguments (
              cls,
             argparse.ArgumentParser parser )
```

Generic class method to add arguments to the cli invocation depending on instancied class.

Reimplemented from lib.QPUBenchClass.QPUBenchClass.

6.2.3.2 capabilities()

```
QPUBenchClassCapability hardware.dwave.DWaveBenchmark.DWaveBenchmark.capabilities (
              self )
```

Detailed implementation:

method to return capabilities of QPU benchmark implementations

Reimplemented from lib.QPUBenchClass.QPUBenchClass.

6.2.3.3 compute()

```
hardware.dwave.DWaveBenchmark.DWaveBenchmark.compute (
              self )
```

Reimplemented from lib.QPUBenchClass.QPUBenchClass.

6.2.3.4 computeIteration()

```
bool hardware.dwave.DWaveBenchmark.DWaveBenchmark.computeIteration (
              self,
             ** kwargs )
```

6.2.3.5 is_available()

```
bool hardware.dwave.DWaveBenchmark.DWaveBenchmark.is_available (
              self,
              qpuArch = None )
```

Detailed implementation:

Check if DWave libraries are available

Reimplemented from lib.QPUBenchClass.QPUBenchClass.

6.2.3.6 postprocess()

```
hardware.dwave.DWaveBenchmark.DWaveBenchmark.postprocess (
              self )
```

Detailed implementation:

print(logfile) instantiation of QPUBenchClass using the collected data

Reimplemented from lib.QPUBenchClass.QPUBenchClass.

6.2.3.7 postprocessIteration()

```
bool hardware.dwave.DWaveBenchmark.DWaveBenchmark.postprocessIteration (
              self,
             ** kwargs )
```

6.2.3.8 preprocess()

```
hardware.dwave.DWaveBenchmark.DWaveBenchmark.preprocess (
              self )
```

Reimplemented from lib.QPUBenchClass.QPUBenchClass.

6.2.3.9 preprocessIteration()

```
bool hardware.dwave.DWaveBenchmark.DWaveBenchmark.preprocessIteration (
              self,
             str bench,
             ** kwargs )
```

6.2.3.10 selectBenchmark()

```
hardware.dwave.DWaveBenchmark.DWaveBenchmark.selectBenchmark (
              self,
             str bench,
             int num,
              qpuArch = None,
             ** kwargs )
```

Reimplemented from lib.QPUBenchClass.QPUBenchClass.

6.2.4 Member Data Documentation

6.2.4.1 capabilitySet [1/2]

```
list hardware.dwave.DWaveBenchmark.DWaveBenchmark.capabilitySet = ['qscore', 'gscore', 'lin2score',
'linsolverany', 'primescore']  [static]
```

set of capabilities for DWave QPUs

6.2.4.2 capabilitySet [2/2]

```
hardware.dwave.DWaveBenchmark.DWaveBenchmark.capabilitySet
```

6.2.4.3 computeRes

```
hardware.dwave.DWaveBenchmark.DWaveBenchmark.computeRes = None  [static]
```

6.2.4.4 currentBench

```
hardware.dwave.DWaveBenchmark.DWaveBenchmark.currentBench = None  [static]
```

6.2.4.5 dat

```
hardware.dwave.DWaveBenchmark.DWaveBenchmark.dat = None  [static]
```

6.2.4.6 dwarch

```
hardware.dwave.DWaveBenchmark.DWaveBenchmark.dwarch = None  [static]
```

6.2.4.7 dwaveDispatch

```
dict hardware.dwave.DWaveBenchmark.DWaveBenchmark.dwaveDispatch  [static]
```

Initial value:

```
=  {'gscore':[setGnDwaveRegister, runDwave, evalGScore],
        'lin2score':[setLin2DwaveRegister, runDwave, evalLin2Score],
        'primescore':[preDwaveMultiplier, useDwaveMultiplier, returnDwavePrime]}
```

6.2.4.8 dwctx

```
hardware.dwave.DWaveBenchmark.DWaveBenchmark.dwctx = QPUBenchContext()  [static]
```

6.2.4.9 dwembedding

```
hardware.dwave.DWaveBenchmark.DWaveBenchmark.dwembedding = None  [static]
```

6.2.4.10 dwsampler

```
hardware.dwave.DWaveBenchmark.DWaveBenchmark.dwsampler = None  [static]
```

6.2.4.11 isInit

```
bool hardware.dwave.DWaveBenchmark.DWaveBenchmark.isInit = False  [static]
```

6.2.4.12 loginfile

```
hardware.dwave.DWaveBenchmark.DWaveBenchmark.loginfile = None  [static]
```

6.2.4.13 myCompute

```
hardware.dwave.DWaveBenchmark.DWaveBenchmark.myCompute = None  [static]
```

6.2.4.14 myPostprocess

```
hardware.dwave.DWaveBenchmark.DWaveBenchmark.myPostprocess = None  [static]
```

6.2.4.15 myPreprocess

```
hardware.dwave.DWaveBenchmark.DWaveBenchmark.myPreprocess = None  [static]
```

6.2.4.16 nIter [1/2]

```
int hardware.dwave.DWaveBenchmark.DWaveBenchmark.nIter = 0  [static]
```

6.2.4.17 nIter [2/2]

```
hardware.dwave.DWaveBenchmark.DWaveBenchmark.nIter
```

6.2.4.18 nqubits [1/2]

```
int hardware.dwave.DWaveBenchmark.DWaveBenchmark.nqubits = -1  [static]
```

6.2.4.19 nqubits [2/2]

```
hardware.dwave.DWaveBenchmark.DWaveBenchmark.nqubits
```

6.2.4.20 postprocessRes

```
hardware.dwave.DWaveBenchmark.DWaveBenchmark.postprocessRes
```

6.2.4.21 preprocessRes

```
hardware.dwave.DWaveBenchmark.DWaveBenchmark.preprocessRes = None  [static]
```

The documentation for this class was generated from the following file:

- hardware/dwave/DWaveBenchmark.py

6.3 IsingModel Class Reference

Public Member Functions

- IsingModel (int size, ModelType mtype, std::vector< int > *buf1, std::vector< int > *buf2)
- void initModel (std::vector< double > *J, std::vector< double > *h)
- int flip (int val)
- double getCurrentEnergy (void)
- double getPreviousEnergy (void)
- void generateCandidateConfiguration (void)
- void selectCurrent (void)
- void discardCurrent (void)
- std::vector< int > * getCurrentConfig (void)

Private Member Functions

- void initRandomConfig (std::vector< int > *configuration)
- double getEnergy (std::vector< int > *isingConfiguration)

Private Attributes

- ModelType isingType = SPIN
- int defaultModelVal = 0
- int isingSize = 0
- std::vector< int > * currentConfig = nullptr
- std::vector< int > * previousConfig = nullptr
- std::vector< double > * myh = nullptr
- std::vector< double > * myJ = nullptr
- double currentEnergy = std::numeric_limits<double>::infinity()
- double previousEnergy = std::numeric_limits<double>::infinity()

6.3.1 Constructor & Destructor Documentation

6.3.1.1 IsingModel()

```
IsingModel::IsingModel (
            int size,
            ModelType mtype,
            std::vector< int > * buf1,
            std::vector< int > * buf2 )
```

6.3.2 Member Function Documentation

6.3.2.1 discardCurrent()

```
void IsingModel::discardCurrent (
            void  )
```

6.3.2.2 flip()

```
int IsingModel::flip (
            int val )
```

6.3.2.3 generateCandidateConfiguration()

```
void IsingModel::generateCandidateConfiguration (
            void  )
```

6.3.2.4 getCurrentConfig()

```
std::vector< int > * IsingModel::getCurrentConfig (
            void  )
```

6.3.2.5 getCurrentEnergy()

```
double IsingModel::getCurrentEnergy (
            void  )
```

6.3.2.6 getEnergy()

```
double IsingModel::getEnergy (
            std::vector< int > * isingConfiguration )  [private]
```

6.3.2.7 getPreviousEnergy()

```
double IsingModel::getPreviousEnergy (
            void  )
```

6.3.2.8 initModel()

```
void IsingModel::initModel (
            std::vector< double > * J,
            std::vector< double > * h )
```

6.3.2.9 initRandomConfig()

```
void IsingModel::initRandomConfig (
            std::vector< int > * configuration )  [private]
```

6.3.2.10 selectCurrent()

```
void IsingModel::selectCurrent (
            void  )
```

6.3.3 Member Data Documentation

6.3.3.1 currentConfig

```
std::vector<int>* IsingModel::currentConfig = nullptr  [private]
```

6.3.3.2 currentEnergy

```
double IsingModel::currentEnergy = std::numeric_limits<double>::infinity()  [private]
```

6.3.3.3 defaultModelVal

```
int IsingModel::defaultModelVal = 0  [private]
```

6.3.3.4 isingSize

```
int IsingModel::isingSize = 0  [private]
```

6.3.3.5 isingType

```
ModelType IsingModel::isingType = SPIN  [private]
```

6.3.3.6 myh

```
std::vector<double>* IsingModel::myh = nullptr  [private]
```

6.3.3.7 myJ

```
std::vector<double>* IsingModel::myJ = nullptr  [private]
```

6.3.3.8 previousConfig

```
std::vector<int>* IsingModel::previousConfig = nullptr  [private]
```

6.3.3.9 previousEnergy

```
double IsingModel::previousEnergy = std::numeric_limits<double>::infinity()  [private]
```

The documentation for this class was generated from the following file:

- hardware/src/simpleSimulatedAnnealing.cpp

6.4 hardware.dwave.DwaveMultiplier.Multiplier Class Reference

Public Member Functions

- __init__ (self, N, Dwave_Machine, sizes=None, chain_strength=2)

  Definition of the Multiplier parameters.
- make_nodes (self, row, col, m, hv, p)

  :param row: int - index of the row :param col: int - index of the column :param m : int - parameter of the Dwave machine :param hv : int - horizontal or vertical parameter of the nice coordinates of Pegasus topology :param p : int - position parameter of the nice coordinates of Pegasus topology
- cfa_nodes (self, row, col)

  :param row: int - index of the row :param col: int - index of the column
- make_subgraph (self, row, col)

  :param row: int - index of the row :param col: int - index of the column
- read_data (self, data)
- cfa_tile_to_graph (self, row, col, tile_0=(0, 0))
- build_graph_complete (self, tile_0=(0, 0))
- plot_multiplier (self, combined_graph=None, tile_0=(0, 0), plot=False, plot_inputs=False)
- Compatibility_w_Real_Graph (self, G, Chains, plot_inputs=True)

Public Attributes

- N
- binary_N
- n
- sizes
- n_row
- n_col
- inputs_to_fix
- factors_nodes
- Dwave_Machine
- H_range
- J_range
- biases
- couplings
- offset
- cfa_plmt
- chain_strength
- Compatibility
- topologie_graph
- coords
- pos
- tile_0

6.4.1 Constructor & Destructor Documentation

6.4.1.1 __init__()

```
hardware.dwave.DwaveMultiplier.Multiplier.__init__ (
              self,
              N,
              Dwave_Machine,
              sizes = None,
              chain_strength = 2 )
```

Definition of the Multiplier parameters.

:param N : int - integer to factorize :param Dwave_Machine : dict - dictionnary of properties of the dwave annealer considered :param sizes : tuple - size of the multiplier (number of rows, number of colums), :param chain_strength : float - chain strength, default value is 2.

Detailed implementation:

1. Load the Ising hamiltonian of the CFA
2. Define the coordinates and subgraphs depending on the Dwave topology.

6.4.2 Member Function Documentation

6.4.2.1 build_graph_complete()

```
hardware.dwave.DwaveMultiplier.Multiplier.build_graph_complete (
              self,
              tile_0 = (0,0) )
```

```
:param tile_0: tuple - position of the first CFA of the multiplier on the Dwave Machine topology. default valu
```

6.4.2.2 cfa_nodes()

```
hardware.dwave.DwaveMultiplier.Multiplier.cfa_nodes (
              self,
              row,
              col )
```

:param row: int - index of the row :param col: int - index of the column

6.4.2.3 cfa_tile_to_graph()

```
hardware.dwave.DwaveMultiplier.Multiplier.cfa_tile_to_graph (
              self,
              row,
              col,
              tile_0 = (0,0) )
```

```
:params row: int - row number of the CFA in the multiplier
:params col: int - column number of the CFA in the multiplier
:params tile_0: tuple - Position of the first CFA on the real machine
```

6.4.2.4 Compatibility_w_Real_Graph()

```
hardware.dwave.DwaveMultiplier.Multiplier.Compatibility_w_Real_Graph (
              self,
              G,
              Chains,
              plot_inputs = True )
```

1. test the compatibility of the Mutliplier graph and the available qubits and couplings of the real Dwave machine.

   :param G: Graph - Multiplier's graph defined in self.plot_multiplier() (without the chains) :param Chains: Graph - Chains's graph defined in self.plot_multiplier() :param plot_inputs: Bool - True: if plot, identifies the qubits' values to fix, purple for 0, green for 1.

   Detailed implementation:

   (a) Available qubits and couplings of the real Dwave machine.
   (b) Identifies if some nodes are not working
   (c) Identifies if some edges are not working
   (d) Non-working nodes in orange, non-working edges in red.

6.4.2.5 make_nodes()

```
hardware.dwave.DwaveMultiplier.Multiplier.make_nodes (
              self,
              row,
              col,
              m,
              hv,
              p )
```

:param row: int - index of the row :param col: int - index of the column :param m : int - parameter of the Dwave machine :param hv : int - horizontal or vertical parameter of the nice coordinates of Pegasus topology :param p : int - position parameter of the nice coordinates of Pegasus topology

6.4.2.6 make_subgraph()

```
hardware.dwave.DwaveMultiplier.Multiplier.make_subgraph (
              self,
              row,
              col )
```

:param row: int - index of the row :param col: int - index of the column

6.4.2.7 plot_multiplier()

```
hardware.dwave.DwaveMultiplier.Multiplier.plot_multiplier (
              self,
              combined_graph = None,
              tile_0 = (0,0),
              plot = False,
              plot_inputs = False )
```

```
:param combined_graph: dict - Bias, couplings, chains and offset of the multiplier graph. result of the functi
:param tile_0: Description
:param plot: Bool - True: plots the multiplier graph onto the machine topology, chains in red.
:param plot_inputs: Bool - True: if plot, identifies the qubits' values to fix, purple for 0, green for 1.
```

6.4.2.8 read_data()

```
hardware.dwave.DwaveMultiplier.Multiplier.read_data (
              self,
              data )
```

1. Reads the biases, couplings, offset and CFA indices from the corresponding .json in the library.

   :param data:

   Detailed implementation:

   (a) Map the a CFA, for a given row and column of the multiplier, onto the topology/coordinates of the Dwave Machine.
   (b) Build the complete graph of the multiplier onto the topology of the Dwave Machine.
   (c) Build the chains between CFAs
   (d) in2-out : Shared qubit.
   (e) in2-cout : chain(s), depending on the topology.
   (f) in1-in1 chain
   (g) enable-enable chain
   (h) cin-cout shared qubit
   (i) cin-cout chain
   (j) Keep track of the qubits corresponding to the factors A(in1) and B(enable).
   (k) Qubit values to impose: first bits of A (in1_0) and B (enable_0) and values of out=N_i and cin=0 and in2=0 fist column and first row.
   (l) Building of the graph of the Multiplier

6.4.3 Member Data Documentation

6.4.3.1 biases

```
hardware.dwave.DwaveMultiplier.Multiplier.biases
```

6.4.3.2 binary_N

```
hardware.dwave.DwaveMultiplier.Multiplier.binary_N
```

6.4.3.3 cfa_plmt

```
hardware.dwave.DwaveMultiplier.Multiplier.cfa_plmt
```

6.4.3.4 chain_strength

```
hardware.dwave.DwaveMultiplier.Multiplier.chain_strength
```

6.4.3.5 Compatibility

```
hardware.dwave.DwaveMultiplier.Multiplier.Compatibility
```

6.4.3.6 coords

hardware.dwave.DwaveMultiplier.Multiplier.coords

6.4.3.7 couplings

hardware.dwave.DwaveMultiplier.Multiplier.couplings

6.4.3.8 Dwave_Machine

hardware.dwave.DwaveMultiplier.Multiplier.Dwave_Machine

6.4.3.9 factors_nodes

hardware.dwave.DwaveMultiplier.Multiplier.factors_nodes

6.4.3.10 H_range

hardware.dwave.DwaveMultiplier.Multiplier.H_range

6.4.3.11 inputs_to_fix

hardware.dwave.DwaveMultiplier.Multiplier.inputs_to_fix

6.4.3.12 J_range

hardware.dwave.DwaveMultiplier.Multiplier.J_range

6.4.3.13 N

hardware.dwave.DwaveMultiplier.Multiplier.N

6.4.3.14 n

hardware.dwave.DwaveMultiplier.Multiplier.n

6.4.3.15 n_col

hardware.dwave.DwaveMultiplier.Multiplier.n_col

6.4.3.16 n_row

hardware.dwave.DwaveMultiplier.Multiplier.n_row

6.4.3.17 offset

hardware.dwave.DwaveMultiplier.Multiplier.offset

6.4.3.18 pos

hardware.dwave.DwaveMultiplier.Multiplier.pos

6.4.3.19 sizes

hardware.dwave.DwaveMultiplier.Multiplier.sizes

6.4.3.20 tile_0

hardware.dwave.DwaveMultiplier.Multiplier.tile_0

6.4.3.21 topologie_graph

hardware.dwave.DwaveMultiplier.Multiplier.topologie_graph

The documentation for this class was generated from the following file:

- hardware/dwave/DwaveMultiplier.py

6.5 hardware.dwave.Run_multiplier.Multiplier_to_Run Class Reference

Public Member Functions

- __init__ (self, Ns, start_tile, sizes, id_Dwave_machine, dwcontext, chain_strength=2, Folder_result_↩ name='_Results', Method_input='flux_biases', Empirical_strength=1000)

  :param Ns : list of int - integers to factorize :param start_tile : tuple - position of the first CFA onto the topology of the Dwave machine :param sizes : tuple - size of the multiplier (number of rows, number of columns) :param id↩ _Dwave_machine : int - number of the Dwave machine in the dictionnary defined in Dwave_machines.py :param chain_strength : float - chain strength, default value is 2.

Public Attributes

- Empirical_strength

6.5.1 Constructor & Destructor Documentation

6.5.1.1 __init__()

```
hardware.dwave.Run_multiplier.Multiplier_to_Run.__init__ (
              self,
              Ns,
              start_tile,
              sizes,
              id_Dwave_machine,
              dwcontext,
              chain_strength = 2,
              Folder_result_name = '_Results',
              Method_input = 'flux_biases',
              Empirical_strength = 1000 )
```

:param Ns : list of int - integers to factorize :param start_tile : tuple - position of the first CFA onto the topology of the Dwave machine :param sizes : tuple - size of the multiplier (number of rows, number of columns) :param id_Dwave_machine : int - number of the Dwave machine in the dictionnary defined in Dwave_machines.py :param chain_strength : float - chain strength, default value is 2.

:param Folder_result_name : str - name of the folder for the results. :param Method_input : str - 'flux biases' or 'API' :param Empirical_strength : int - value of the empirical strength if Method_input is 'flux biases', default value is 1000.

Detailed implementation:

1. First integer to fatorize - check the compatibility with the real machine:
2. if Multiplier compatible with the machine, run all the factorization problems.
3. Prepare the problem for the real machine
4. Definition of the inputs

Detailed implementation:

1. Check of the reduced ising
2. Run and save the results

6.5.2 Member Data Documentation

6.5.2.1 Empirical_strength

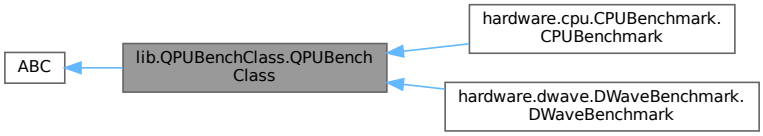
```
hardware.dwave.Run_multiplier.Multiplier_to_Run.Empirical_strength
```

The documentation for this class was generated from the following file:

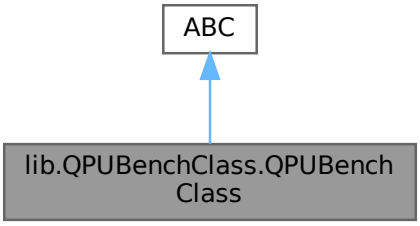
- hardware/dwave/Run_multiplier.py

6.6 lib.QPUBenchClass.QPUBenchClass Class Reference

Inheritance diagram for lib.QPUBenchClass.QPUBenchClass:



Collaboration diagram for lib.QPUBenchClass.QPUBenchClass:



Public Member Functions

- QPUBenchClassCapability capabilities (self)
- selectBenchmark (self, str bench, ∗∗kwargs)
- Any preprocess (self, Any data, ∗∗kwargs)
- Any compute (self, Any data, ∗∗kwargs)
- Any postprocess (self, Any data, ∗∗kwargs)
- bool is_available (self)

  Check if the QPU is available at runtime.
- add_arguments (cls, argparse.ArgumentParser parser)

  Generic class method to add arguments to the cli invocation depending on instancied class.

6.6.1 Member Function Documentation

6.6.1.1 add_arguments()

```
lib.QPUBenchClass.QPUBenchClass.add_arguments (
              cls,
            argparse.ArgumentParser parser )
```

Generic class method to add arguments to the cli invocation depending on instancied class.

Reimplemented in hardware.cpu.CPUBenchmark.CPUBenchmark, and hardware.dwave.DWaveBenchmark.DWaveBenchmark.

6.6.1.2 capabilities()

```
QPUBenchClassCapability lib.QPUBenchClass.QPUBenchClass.capabilities (
              self )
```

Reimplemented in hardware.cpu.CPUBenchmark.CPUBenchmark, and hardware.dwave.DWaveBenchmark.DWaveBenchmark.

6.6.1.3 compute()

```
Any lib.QPUBenchClass.QPUBenchClass.compute (
              self,
             Any data,
             ** kwargs )
```

Reimplemented in hardware.cpu.CPUBenchmark.CPUBenchmark, and hardware.dwave.DWaveBenchmark.DWaveBenchmark.

6.6.1.4 is_available()

```
bool lib.QPUBenchClass.QPUBenchClass.is_available (
              self )
```

Check if the QPU is available at runtime.

Reimplemented in hardware.cpu.CPUBenchmark.CPUBenchmark, and hardware.dwave.DWaveBenchmark.DWaveBenchmark.

6.6.1.5 postprocess()

```
Any lib.QPUBenchClass.QPUBenchClass.postprocess (
              self,
             Any data,
             ** kwargs )
```

Reimplemented in hardware.cpu.CPUBenchmark.CPUBenchmark, and hardware.dwave.DWaveBenchmark.DWaveBenchmark.

6.6.1.6 preprocess()

```
Any lib.QPUBenchClass.QPUBenchClass.preprocess (
              self,
             Any data,
             ** kwargs )
```

Reimplemented in hardware.cpu.CPUBenchmark.CPUBenchmark, and hardware.dwave.DWaveBenchmark.DWaveBenchmark.

6.6.1.7 selectBenchmark()

```
lib.QPUBenchClass.QPUBenchClass.selectBenchmark (
              self,
             str bench,
             ** kwargs )
```

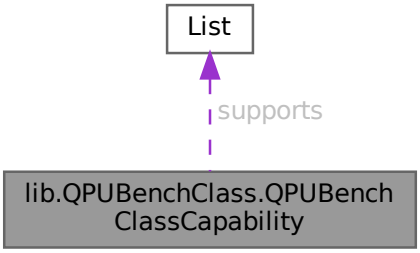
Reimplemented in hardware.cpu.CPUBenchmark.CPUBenchmark, and hardware.dwave.DWaveBenchmark.DWaveBenchmark.

The documentation for this class was generated from the following file:

- lib/QPUBenchClass.py

6.7 lib.QPUBenchClass.QPUBenchClassCapability Class Reference

Collaboration diagram for lib.QPUBenchClass.QPUBenchClassCapability:



Static Public Attributes

- str name
- List supports [str]

6.7.1 Member Data Documentation

6.7.1.1 name

```
str lib.QPUBenchClass.QPUBenchClassCapability.name  [static]
```

6.7.1.2 supports

```
List lib.QPUBenchClass.QPUBenchClassCapability.supports [str]  [static]
```

The documentation for this class was generated from the following file:

- lib/QPUBenchClass.py

6.8 lib.QPUBenchClass.QPUBenchContext Class Reference

Static Public Attributes

- int nIter = -1

  Iteration number on the current benchmark score.
- int dat = 0

  secondary value relevant for the benchmark, versatile use whose signification depends on the benchmark
- int target = 100000

  target value (optional)
- int proba = -1

  evaluated probability of outcome for the given computing
- computeRes = None

  bare bone results from QPU computing (before formatting)
- currentBench = None

  name of the current benchmark (e.g.
- arch = None

  architecture name
- subarch = None

  subarchitecture name (within architecture what specific QPU is used)
- formatedSamples = None

  current samples (outcome for GPU, but formatted and selected)
- probaDist = None

  list of probability for statistical analysis (depending on benchmark program)
- list distVectors = [[ ],[ ],[ ]]

6.8.1 Member Data Documentation

6.8.1.1 arch

```
lib.QPUBenchClass.QPUBenchContext.arch = None  [static]
```

architecture name

6.8.1.2 computeRes

```
lib.QPUBenchClass.QPUBenchContext.computeRes = None  [static]
```

bare bone results from QPU computing (before formatting)

6.8.1.3 currentBench

```
lib.QPUBenchClass.QPUBenchContext.currentBench = None  [static]
```

name of the current benchmark (e.g.

'gscore')

6.8.1.4 dat

```
int lib.QPUBenchClass.QPUBenchContext.dat = 0  [static]
```

secondary value relevant for the benchmark, versatile use whose signification depends on the benchmark

6.8.1.5 distVectors

```
list lib.QPUBenchClass.QPUBenchContext.distVectors = [[],[],[]]  [static]
```

6.8.1.6 formatedSamples

```
lib.QPUBenchClass.QPUBenchContext.formatedSamples = None  [static]
```

current samples (outcome for GPU, but formatted and selected)

6.8.1.7 nIter

```
int lib.QPUBenchClass.QPUBenchContext.nIter = -1  [static]
```

Iteration number on the current benchmark score.

6.8.1.8 proba

```
int lib.QPUBenchClass.QPUBenchContext.proba = -1  [static]
```

evaluated probability of outcome for the given computing

6.8.1.9 probaDist

```
lib.QPUBenchClass.QPUBenchContext.probaDist = None  [static]
```

list of probability for statistical analysis (depending on benchmark program)

6.8.1.10 subarch

```
lib.QPUBenchClass.QPUBenchContext.subarch = None  [static]
```

subarchitecture name (within architecture what specific QPU is used)

6.8.1.11 target

```
int lib.QPUBenchClass.QPUBenchContext.target = 100000  [static]
```

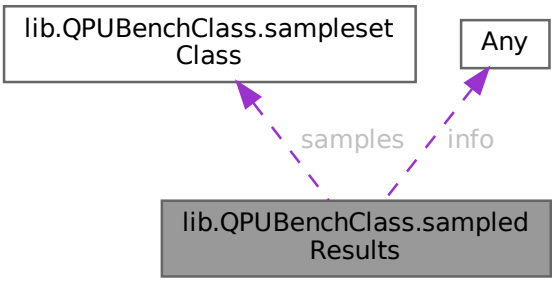
target value (optional)

The documentation for this class was generated from the following file:

- lib/QPUBenchClass.py

6.9 lib.QPUBenchClass.sampledResults Class Reference

Collaboration diagram for lib.QPUBenchClass.sampledResults:



Static Public Attributes

- samplesetClass samples = None
- int num = -1
- int totNum = -1
- float energyCost = 1E6
- int numQubits = -1
- Any info = None

6.9.1 Member Data Documentation

6.9.1.1 energyCost

```
float lib.QPUBenchClass.sampledResults.energyCost = 1E6  [static]
```

6.9.1.2 info

```
Any lib.QPUBenchClass.sampledResults.info = None  [static]
```

6.9.1.3 num

```
int lib.QPUBenchClass.sampledResults.num = -1  [static]
```

6.9.1.4 numQubits

```
int lib.QPUBenchClass.sampledResults.numQubits = -1  [static]
```

6.9.1.5 samples

```
samplesetClass lib.QPUBenchClass.sampledResults.samples = None  [static]
```

6.9.1.6 totNum

```
int lib.QPUBenchClass.sampledResults.totNum = -1  [static]
```

The documentation for this class was generated from the following file:

- lib/QPUBenchClass.py

6.10 lib.QPUBenchClass.samplesetClass Class Reference

Static Public Attributes

- np vecVal .ndarray
- float energy
- int occurences = 1

6.10.1 Member Data Documentation

6.10.1.1 energy

```
float lib.QPUBenchClass.samplesetClass.energy  [static]
```

6.10.1.2 occurences

```
int lib.QPUBenchClass.samplesetClass.occurences = 1  [static]
```

6.10.1.3 vecVal

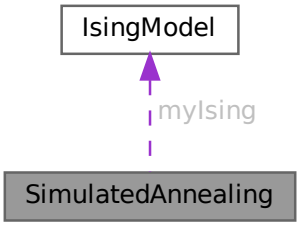
```
np lib.QPUBenchClass.samplesetClass.vecVal .ndarray  [static]
```

The documentation for this class was generated from the following file:

- lib/QPUBenchClass.py

6.11 SimulatedAnnealing Class Reference

Collaboration diagram for SimulatedAnnealing:



Public Member Functions

- SimulatedAnnealing (int size, ModelType mtype)
- void initCoupling (std::vector< double > *J, std::vector< double > *h)
- void run (void)
- void setInitialTemperature (double temp)
- std::vector< int > * getCurrentConfig (void)
- void setScale (double scale)
- void setSwipeNumber (int nswipe)
- void setCurrentAsBest (void)
- std::vector< int > result (void)
- double getBestEnergy (void)

Private Attributes

- IsingModel * myIsing = nullptr
- int Isize = 0
- double initTemperature = 100.0
- double endingTemperature = 1e-6
- double deltaTemperature = 0.95
- double scale = 10.0
- int numSwipe = 200
- std::array< std::vector< int >, 3 > configbuf
- double bestEnergy = std::numeric_limits<double>::infinity()

6.11.1 Constructor & Destructor Documentation

6.11.1.1 SimulatedAnnealing()

```
SimulatedAnnealing::SimulatedAnnealing (
            int size,
            ModelType mtype )
```

6.11.2 Member Function Documentation

6.11.2.1 getBestEnergy()

```
double SimulatedAnnealing::getBestEnergy (
            void  )
```

6.11.2.2 getCurrentConfig()

```
std::vector< int > * SimulatedAnnealing::getCurrentConfig (
            void  )
```

6.11.2.3 initCoupling()

```
void SimulatedAnnealing::initCoupling (
            std::vector< double > * J,
            std::vector< double > * h )
```

6.11.2.4 result()

```
std::vector< int > SimulatedAnnealing::result (
            void  )
```

6.11.2.5 run()

```
void SimulatedAnnealing::run (
            void  )
```

6.11.2.6 setCurrentAsBest()

```
void SimulatedAnnealing::setCurrentAsBest (
            void  )
```

6.11.2.7 setInitialTemperature()

```
void SimulatedAnnealing::setInitialTemperature (
            double temp )
```

6.11.2.8 setScale()

```
void SimulatedAnnealing::setScale (
            double scale )
```

6.11.2.9 setSwipeNumber()

```
void SimulatedAnnealing::setSwipeNumber (
            int nswipe )
```

6.11.3 Member Data Documentation

6.11.3.1 bestEnergy

```
double SimulatedAnnealing::bestEnergy = std::numeric_limits<double>::infinity()  [private]
```

6.11.3.2 configbuf

```
std::array<std::vector<int>,3> SimulatedAnnealing::configbuf  [private]
```

6.11.3.3 deltaTemperature

```
double SimulatedAnnealing::deltaTemperature = 0.95  [private]
```

6.11.3.4 endingTemperature

```
double SimulatedAnnealing::endingTemperature = 1e-6  [private]
```

6.11.3.5 initTemperature

```
double SimulatedAnnealing::initTemperature = 100.0  [private]
```

6.11.3.6 Isize

```
int SimulatedAnnealing::Isize = 0  [private]
```

6.11.3.7 myIsing

```
IsingModel* SimulatedAnnealing::myIsing = nullptr  [private]
```

6.11.3.8 numSwipe

```
int SimulatedAnnealing::numSwipe = 200  [private]
```

6.11.3.9 scale

```
double SimulatedAnnealing::scale = 10.0  [private]
```

The documentation for this class was generated from the following file:

- hardware/src/simpleSimulatedAnnealing.cpp

Chapter 7

File Documentation

7.1 BACQBenchmark.py File Reference

Namespaces

- namespace BACQBenchmark

Functions

- BACQBenchmark.main ()

  Main implementation of BACQ Benchmark

Variables

- dict BACQBenchmark.backendMap
- dict BACQBenchmark.BACKENDS = {}

  BACKENDS is a list of available backends.
- str BACQBenchmark.modpath = f"hardware.{backendName}.{moduleName}"

  try and import relevant backend library
- BACQBenchmark.module = importlib.import_module(modpath, package=__name__)
- BACQBenchmark.backendClass = getattr(module, className)

  get class within module

7.2 benchmarks/gscore/GnBenchLib.py File Reference

Namespaces

- namespace GnBenchLib

  GnBenchLib is the generic (QPU independent) function sets for implementing the GScore.

Generated by Doxygen

Functions

- GnBenchLib.maxValH (h)

  Returns extremal value within a given vector.
- GnBenchLib.GnQUBO (n, Verbose=False)

  Generate Gn QUBO matrix.
- GnBenchLib.edgestosummitGn (n)

  Compute the corresponding sets between Gn nodes and Gn edges Input: value of n for Gn Output: double vector for corresponding set between edges and nodes.
- GnBenchLib.simpleQUBO (E)

  generate simplified QUBO matrix with values in {-1,0,1} input: numpy matrix for QUBO output: simplified QUBO matrix (numpy)
- GnBenchLib.GnMISadjacency (n)

  define the adjacency matrix for MIS solving
  input: n value for Gn output: adjacency matrix for the MIS formulation of Gn
- GnBenchLib.hammingDist (n, sample)

  Compute the Haming distance from optimal solution of Gn series in standard numbering of edges to the actual found solution Input: value of n for Gn , vector candidate solution Output: integer= Hamming distance= number of discrepencies in the bit string of the provided vector.
- GnBenchLib.lowestHammingDist (n, sample)

  Search the set of samples with the lowest Haming distance to the optimal.
- GnBenchLib.brokenConstraints (n, sample)

  Compute the number of broken constraints in the candidate vector solution.
- GnBenchLib.lowestBrokenConstraints (n, sample)

  Search the lowest set of samples with the lowest number of broken constraints.
- GnBenchLib.gnScore (nfailed, proba, sample, redux=0.0)

  Compute GScore from.
- GnBenchLib.successProbaGn (n, sampledResults samples)

  Return an estimator of the success probability for the Gn optimization outcome.
- GnBenchLib.lowestCost (n, samples)

  Search minimal cost from samples for Gn.
- GnBenchLib.evalGScore (ctx)
- (float, int) GnBenchLib.simplification_score (np.ndarray A, np.ndarray A_prime, float lambda_pen=1)

  Evaluate graph simplification in terms of equivalent number of nodes.
- float GnBenchLib.scoreRedux (int n, np.ndarray A, np.ndarray Ap)

Variables

- GnBenchLib.root = Path(__file__).resolve().parents[2]
- GnBenchLib.n = int(sys.argv[1])
- GnBenchLib.myQ
- GnBenchLib.numval
- GnBenchLib.N = len(myQ)
- GnBenchLib.h
- GnBenchLib.J
- GnBenchLib.end
- GnBenchLib.simpQ = simpleQUBO(myQ)
- GnBenchLib.simpIsingh
- GnBenchLib.simpIsingJ
- GnBenchLib.max = maxValH(h)
- int GnBenchLib.val = 0
- GnBenchLib.myl = len(h)
- bool GnBenchLib.tbc = False

- GnBenchLib.A = GnMISadjacency(n)
- GnBenchLib.A2_3d
- GnBenchLib.A2_2d
- float GnBenchLib.redux2 = scoreRedux(2, A, A2_2d)
- GnBenchLib.pseudosample = sampledResults(np.array([[1,1,1,1,1,1,1,1,1,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0]]),1,1,1E6,27,None
- float GnBenchLib.redux3 = scoreRedux(2, A, A2_3d)

7.3 benchmarks/linscore/LinBench2.py File Reference

Namespaces

- namespace benchmarks
- namespace benchmarks.linscore
- namespace benchmarks.linscore.LinBench2
- namespace LinBench2

  LinBench2 is the implementation of function sets for implementing Lin2Bench and LinNBench benchmarks.

Functions

- benchmarks.linscore.LinBench2.generateAB (Xmat, dyn=5, sign=1)

  Generate matrix A and vector B for Ax+B=0 equation from the planned solution Xmat.
- benchmarks.linscore.LinBench2.find2bitSolutions (Qmat)

  Find 2's complement solution.
- benchmarks.linscore.LinBench2.truncate2bits (Amat, Bmat)

  Generate binary matrices (2's complement encoding) from matrix A and vector B.
- benchmarks.linscore.LinBench2.generateXrandom (myr, dyn=20)

  Generate a random solution vector of size myr and range in dyn inputs: myr (int, size of vector solution), dyn (int, range for random number generation)
- benchmarks.linscore.LinBench2.valcost (vecX, numbit, h, J)

  Gives the minimum cost (optimal target) for the QUBO problem associated with AX+C=0.
- benchmarks.linscore.LinBench2.LinProb (myrange, bitnum)

  LinProb is the frontend function to implement the Lin∗Bench benchmark LinProb provides a random X vector solution and the problem instance that corresponds A matrix and B vector so that AX+B=0.
- benchmarks.linscore.LinBench2.Lin2Prob (myrange)

  Lin2Prob is the frontend function to implement the Lin2Bench benchmark It provides the A matrix, B vector and X solution vector with $X \in \{0,1\}^n$.
- benchmarks.linscore.LinBench2.QUBOLin2Bench (myrange)

  QUBOLin2Bench is one of the backend function to implement the Lin2Bench benchmark It gives a QUBO formulation of an instance of the problem to solve.
- benchmarks.linscore.LinBench2.successProbaLin2 (ctx, mysample)

  sucessProbaLin2 implements the evaluation of the success probability on one given instance of a problem it will rely on the already available optimal score computed in the previous function (QUBOLin2Bench)
- benchmarks.linscore.LinBench2.fitSigmoid (ctx, filename=None)

  fitSigmoid= fit a sigmoide function with the set of values $f(x) = \frac{1}{1+e^{-\lambda(x-x_0)}}$ *At the end the relevant value will be* x_0 *with its evaluated standard deviation But we also return the \lambda value with its standard deviation*
- benchmarks.linscore.LinBench2.evalLin2Score (ctx)

  evalLin2Score function implements the Lin2Bench score evaluation in an incremental fashion To be set as post-processing method according to QPU call and management
- benchmarks.linscore.LinBench2.main ()

  Main program: unitary and integration tests (when not used as a library)

Variables

- benchmarks.linscore.LinBench2.root = Path(__file__).resolve().parents[2]

7.4 benchmarks/primescore/prime-factorization.py File Reference

Namespaces

- namespace prime-factorization

Functions

- prime-factorization.listupdate (mylist, myelement)
- prime-factorization.HobofyPrime (N, nbits, pbits)
- prime-factorization.bestConfig (numbit, hobo)
- prime-factorization.quadratize (nvar, initlist)
- prime-factorization.defMval (mylist, myM)
- prime-factorization.reducePolynomial (numvar, myDinom)
- prime-factorization.exportQUBOdict (mylist)

Variables

- int prime-factorization.tofactorize = 15
- prime-factorization.mylist2
- prime-factorization.mylist3
- prime-factorization.mylist4
- prime-factorization.list34 = mylist3+mylist4
- prime-factorization.totvar
- prime-factorization.newlist2
- prime-factorization.valuedlist = defMval(newlist2,200.0)
- prime-factorization.finalList2 = reducePolynomial(totvar,mylist2+valuedlist)
- prime-factorization.myh
- prime-factorization.myJ

7.5 benchmarks/__init__.py File Reference

7.6 benchmarks/linscore/__init__.py File Reference

7.7 hardware/__init__.py File Reference

7.8 hardware/cpu/__init__.py File Reference

7.9 hardware/dwave/__init__.py File Reference

7.10 lib/__init__.py File Reference

7.11 hardware/cpu/CPUBenchmark.py File Reference

Classes

- class hardware.cpu.CPUBenchmark.CPUBenchmark

Namespaces

- namespace hardware
- namespace hardware.cpu
- namespace hardware.cpu.CPUBenchmark

Functions

- ctypes.CDLL hardware.cpu.CPUBenchmark.load_library (str lib_name, Optional[Path] relative_path=None)

  Sideload a library in a multi-plateform way.
- Path hardware.cpu.CPUBenchmark.get_lib_path (str lib_name)

  Return library path according to underlying OS.
- hardware.cpu.CPUBenchmark.selectCPUheuristic (algoType='sa')

  Select base algorithm for CPU base benchmark.
- hardware.cpu.CPUBenchmark.runSAmt_py (np.ndarray J, np.ndarray h, int mytype, int nsamp)

  calling C function runSAmt from Python3
- hardware.cpu.CPUBenchmark.setGnSARegister (niter, logfile=None, Verbose=False)

  Preprocessing for Gn series: set J and h so that runSAmt can use them to solve Gn.
- hardware.cpu.CPUBenchmark.lowestSamples (samples)
- hardware.cpu.CPUBenchmark.formatSelectSample (sampleset, lowest=True)

  Transform CPU pseudo samples to sampledResults class input: sample set output: sampledResults samples.
- list[list[int]] hardware.cpu.CPUBenchmark.factorize_all (int n, int time_limit_sec=60)

  Python bridge to the C++ factorize_all function.
- hardware.cpu.CPUBenchmark.runFactorizeCPU (nmin, nmax, dum, samplenb)

  Use CPU function using a sieve algorithm to sample the benchmark
- hardware.cpu.CPUBenchmark.preprocessPrime (n)

  dummy preprocess for prime factorization
- hardware.cpu.CPUBenchmark.postprocessPrime (ctx)

  dummy postprocess for prime factorization.
- hardware.cpu.CPUBenchmark.preprocessLin2 (n)
- hardware.cpu.CPUBenchmark.dummyLin2cpu (nmin, nmax, dum, samplenb)
- hardware.cpu.CPUBenchmark.postprocessLin2 (ctx)

  dummy postprocess for lin1score.

Variables

- hardware.cpu.CPUBenchmark.root = Path(__file__).resolve().parents[2]
- Path hardware.cpu.CPUBenchmark.lib_path = get_lib_path("BacqBench")
- hardware.cpu.CPUBenchmark.lib = ctypes.CDLL(str(lib_path))
- dict hardware.cpu.CPUBenchmark.CPUAlgoTypes = {'sa', 'sqa'}

  'sa' for simulated annealing, and 'sqa' for simulated quantum annealing (Monte-Carlo simulation)
- int hardware.cpu.CPUBenchmark.BINARY = 0
- int hardware.cpu.CPUBenchmark.SPIN = 1
- hardware.cpu.CPUBenchmark.argtypes
- hardware.cpu.CPUBenchmark.dtype
- hardware.cpu.CPUBenchmark.float64
- hardware.cpu.CPUBenchmark.flags
- hardware.cpu.CPUBenchmark.int32
- hardware.cpu.CPUBenchmark.restype
- list[list[int]] hardware.cpu.CPUBenchmark.factors = factorize_all(20, time_limit_sec=30)

Generated by Doxygen

7.12 hardware/dwave/Dwave_machines.py File Reference

Namespaces

- namespace hardware
- namespace hardware.dwave
- namespace hardware.dwave.Dwave_machines

Variables

- dict hardware.dwave.Dwave_machines.Dwave_Machines_dict

7.13 hardware/dwave/DWaveBenchmark.py File Reference

Classes

- class hardware.dwave.DWaveBenchmark.DWaveBenchmark

Namespaces

- namespace hardware
- namespace hardware.dwave
- namespace hardware.dwave.DWaveBenchmark
- namespace DWaveBenchmark

  This is the implementation of function sets for implementing DWave QPU particular access (clean Ocean SDK installed).

Functions

- hardware.dwave.DWaveBenchmark.selectDWaveQPU (archType='pegasus')

  Select DWave QPU by using architecture and requiring an available QPU with the given architecture.
- hardware.dwave.DWaveBenchmark.generate_embedding (BinaryQuadraticModel bqm, DWaveSampler sampler, list[int] seeds, bool return_num_qubits=False)

  Generate embeddings for the bqm on the sample with the given seeds.
- BinaryQuadraticModel hardware.dwave.DWaveBenchmark.setGnDwaveRegister (mysampler, niter, ctx, logfile=None, Verbose=False)

  setGnDwaveRegister set the DWave register to solve the Gn problem
- BinaryQuadraticModel hardware.dwave.DWaveBenchmark.setLin2DwaveRegister (mysampler, niter, ctx, logfile=None, Verbose=False)

  setLin2DwaveRegister set the DWave register to solve the Lin2Bench problem
- hardware.dwave.DWaveBenchmark.runDwave (sampler=None, embedding=None, model=None)

  Use DWave procedures to solve the problem on the QPU You can refer to DWave ocean documentation The principle is to use the "better" fixed minor-embedding found by the use of generate_embedding function input: valid sampler class refering to an actual DWave QPU, fixed embedding, and a QUBO/Ising model to run on the QPU output: Dwave ocean sample class.
- hardware.dwave.DWaveBenchmark.sampleSetDwave (myresponse=None)
- hardware.dwave.DWaveBenchmark.formatSelectDwSample (dwsampleset, lowest=True)

  Transform DWave samples from Sample Ocean class to sampledResults class input: DW sample output: sampled↩ Results samples.

Variables

- dict hardware.dwave.DWaveBenchmark.DWaveArchTypes = {'pegasus', 'zephyr'}

  DWave Available Architectures= make it evolve through the commercial availability of QPUs.

7.14 hardware/dwave/DwaveMultiplier.py File Reference

Classes

- class hardware.dwave.DwaveMultiplier.Multiplier

Namespaces

- namespace hardware
- namespace hardware.dwave
- namespace hardware.dwave.DwaveMultiplier

7.15 hardware/dwave/Run_multiplier.py File Reference

Classes

- class hardware.dwave.Run_multiplier.Multiplier_to_Run

Namespaces

- namespace hardware
- namespace hardware.dwave
- namespace hardware.dwave.Run_multiplier

Functions

- hardware.dwave.Run_multiplier.save_dwave_txt (sampleset, filepath, title="RESULTS")
- hardware.dwave.Run_multiplier.returnDwavePrime (ctx)
- hardware.dwave.Run_multiplier.preDwaveMultiplier (sampler, ni, ctx)
- hardware.dwave.Run_multiplier.useDwaveMultiplier (sampler, ni, ctx)

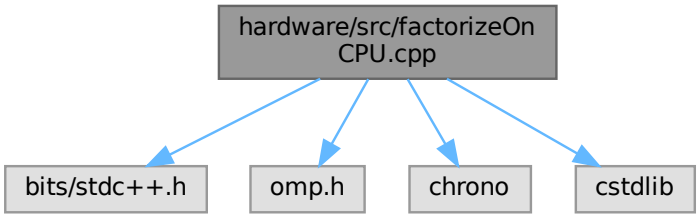
  Function to run the benchmark on specified integer Input:

7.16 hardware/src/factorizeOnCPU.cpp File Reference

Prime factorization using linear sieve algorithm Provides the implementation of function sets for implementing CPU/QPU comparison benchmarks for prime factorization using the linear sieve algorithm.
Dependencies are:

```
#include <bits/stdc++.h>
#include <omp.h>
#include <chrono>
#include <cstdlib>
```

Include dependency graph for factorizeOnCPU.cpp:



Typedefs

- using ll = long long

Functions

- vector< vector< int > > factorize_all (int n, int time_limit_sec=60)
- int factorize_all_c (int n, int time_limit_sec, int **out_data, int **out_lengths, int *out_num_rows)
- void free_factorization (int *data, int *lengths)

7.16.1 Detailed Description

Prime factorization using linear sieve algorithm Provides the implementation of function sets for implementing CPU/QPU comparison benchmarks for prime factorization using the linear sieve algorithm.
Dependencies are:

- stdc++ (C++20)
- openmp (recomended)

Copyright

(c) CEA LIST, QuLIST team
Distributed under EU Public Licence (EUPL)

```
@article{10.1145/359657.359660,
author = {Gries, David and Misra, Jayadev},
title = {A linear sieve algorithm for finding prime numbers},
year = {1978},
issue_date = {Dec. 1978},
publisher = {Association for Computing Machinery},
address = {New York, NY, USA},
volume = {21},
number = {12},
issn = {0001-0782},
url = {https://doi.org/10.1145/359657.359660},
doi = {10.1145/359657.359660},
abstract = {A new algorithm is presented for finding all primes between 2 and n. The algorithm execu
journal = {Commun. ACM},
month = dec,
pages = {999-1003},
numpages = {5},
keywords = {primes, data structures, algorithms}}
```

7.16.2 Typedef Documentation

7.16.2.1 ll

```
using ll = long long
```

7.16.3 Function Documentation

7.16.3.1 factorize_all()

```
vector< vector< int > > factorize_all (
            int n,
            int time_limit_sec = 60 )
```

7.16.3.2 factorize_all_c()

```
int factorize_all_c (
            int n,
            int time_limit_sec,
            int ** out_data,
            int ** out_lengths,
            int * out_num_rows )
```

7.16.3.3 free_factorization()

```
void free_factorization (
            int * data,
            int * lengths )
```

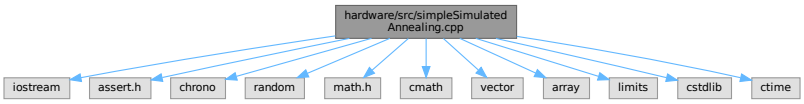
7.17 hardware/src/simpleSimulatedAnnealing.cpp File Reference

Optimization heuristics implemented in C++ especially for CPU/QPU comparison Provides the implementation of function sets for implementing CPU/QPU comparison benchmarks for optimization heuristics. It provides both a usual binary background and a Ising implementation as well.
Dependencies are:

```
#include <iostream>
#include <assert.h>
#include <chrono>
```

```
#include <random>
#include <math.h>
#include <cmath>
#include <vector>
#include <array>
#include <limits>
#include <cstdlib>
#include <ctime>
```

Include dependency graph for simpleSimulatedAnnealing.cpp:



Classes

- class IsingModel
- class SimulatedAnnealing

Macros

- #define OMP_PARALLEL

Enumerations

- enum ModelType { BINARY , SPIN }

Functions

- double uniform (void)
- int uniformDist (int maxRand)

7.17.1 Detailed Description

Optimization heuristics implemented in C++ especially for CPU/QPU comparison Provides the implementation of function sets for implementing CPU/QPU comparison benchmarks for optimization heuristics. It provides both a usual binary background and a Ising implementation as well.
Dependencies are:

- stdc++ (C++20)
- openmp (recomended)

Copyright

(c) CEA LIST, QuLIST team
Distributed under EU Public Licence (EUPL)

```
See Marc C. Robini, Thierry Rastello, Isabelle E. Magnin
"Simulated Annealing, Acceleration Techniques, and Image Restoration"
IEEE Transactions on Image Processing, Vol. 8, No. 10, 1999, pp. 1374-1387.
For justifications of the implementation (geometric cooling)
```

7.17.2 Macro Definition Documentation

7.17.2.1 OMP_PARALLEL

```
#define OMP_PARALLEL
```

7.17.3 Enumeration Type Documentation

7.17.3.1 ModelType

```
enum ModelType
```

Enumerator

BINARY	
SPIN	

7.17.4 Function Documentation

7.17.4.1 uniform()

```
double uniform (
            void  )
```

7.17.4.2 uniformDist()

```
int uniformDist (
            int maxRand )
```

7.18 lib/linear_algebra_ising.py File Reference

Namespaces

- namespace lib
- namespace lib.linear_algebra_ising

Functions

- lib.linear_algebra_ising.QUBOfy (Amat, Bvect, bitsize, sig=1)
  transform Ax+B=0 linear system into a QUBO cost function
- lib.linear_algebra_ising.QUBOtoIsing (myQUBO)
- lib.linear_algebra_ising.IsingtoQUBO (myH, myJ)
- lib.linear_algebra_ising.QUBOoutput (myQUBO)
- lib.linear_algebra_ising.IsingOutput (myH, myJ)
- lib.linear_algebra_ising.getHforDimod (myh)
- lib.linear_algebra_ising.getHQUBOforDimod (myJ)
- lib.linear_algebra_ising.getJforDimod (myJ)
- lib.linear_algebra_ising.maxValQUBO (myQUBO)
- lib.linear_algebra_ising.maxValIsing (myH, myJ)
- lib.linear_algebra_ising.duplicateQUBO (myQUBO, num, cs, wantDiag=True, isLoop=True)
- lib.linear_algebra_ising.duplicateIsing (myH, myJ, num, cs, isLoop=True)

Variables

- lib.linear_algebra_ising.A
- lib.linear_algebra_ising.B
- lib.linear_algebra_ising.numbit
- lib.linear_algebra_ising.sig
- lib.linear_algebra_ising.Qg = np.array([[1,0],[0,-1]])
- lib.linear_algebra_ising.Qproblem = QUBOfy(A,B,numbit,sig)
- lib.linear_algebra_ising.Iproblem = QUBOtoIsing(Qproblem)
- lib.linear_algebra_ising.Q2 = IsingtoQUBO(Iproblem[0],Iproblem[1])
- lib.linear_algebra_ising.QLp = duplicateQUBO(Qproblem, 2, 0.0, True)
- lib.linear_algebra_ising.ILp = duplicateIsing(Iproblem[0],Iproblem[1],3, 18)

7.19 lib/mylinearequations.py File Reference

Namespaces

- namespace lib
- namespace lib.mylinearequations

Functions

- lib.mylinearequations.eq3_3_u (A, B, numbit, sig)
- lib.mylinearequations.eq2_2_u (A, B, numbit, sig)
- lib.mylinearequations.eq2_2_s (A, B, numbit, sig)
- lib.mylinearequations.eq2_3_u (A, B, numbit, sig)
- lib.mylinearequations.eq2_3_s (A, B, numbit, sig)
- lib.mylinearequations.eq3_2_u (A, B, numbit, sig)
- lib.mylinearequations.eq3_2_2_1_u (A, B, numbit, sig)
- lib.mylinearequations.g_1_qubo (Q)
- lib.mylinearequations.g_1_simple_qubo (Q)

7.20 lib/QPUBenchClass.py File Reference

Classes

- class lib.QPUBenchClass.QPUBenchClassCapability
- class lib.QPUBenchClass.QPUBenchContext
- class lib.QPUBenchClass.samplesetClass
- class lib.QPUBenchClass.sampledResults
- class lib.QPUBenchClass.QPUBenchClass

Namespaces

- namespace lib
- namespace lib.QPUBenchClass
- namespace QPUBenchClass

  QPUBenchClass is the set of reference classes for project BACQ refering to management of multiple benchmark scores and multiple targets.

Index

__init__
  hardware.cpu.CPUBenchmark.CPUBenchmark, 47
  hardware.dwave.DWaveBenchmark.DWaveBenchmark, 53
  hardware.dwave.DwaveMultiplier.Multiplier, 62
  hardware.dwave.Run_multiplier.Multiplier_to_Run, 67

A
  GnBenchLib, 21
  lib.linear_algebra_ising, 38
A2_2d
  GnBenchLib, 21
A2_3d
  GnBenchLib, 22
add_arguments
  hardware.cpu.CPUBenchmark.CPUBenchmark, 47
  hardware.dwave.DWaveBenchmark.DWaveBenchmark, 54
  lib.QPUBenchClass.QPUBenchClass, 68
arch
  lib.QPUBenchClass.QPUBenchContext, 71
argtypes
  hardware.cpu.CPUBenchmark, 30

B
  lib.linear_algebra_ising, 38
backendClass
  BACQBenchmark, 10
backendMap
  BACQBenchmark, 10
BACKENDS
  BACQBenchmark, 10
BACQBenchmark, 9
  backendClass, 10
  backendMap, 10
  BACKENDS, 10
  main, 9
  modpath, 10
  module, 10
BACQBenchmark.py, 79
benchmarks, 11
benchmarks.linscore, 11
benchmarks.linscore.LinBench2, 11
  evalLin2Score, 12
  find2bitSolutions, 12
  fitSigmoid, 12
  generateAB, 12
  generateXrandom, 13
  Lin2Prob, 13
  LinProb, 13
  main, 13
  QUBOLin2Bench, 14
  root, 15
  successProbaLin2, 14
  truncate2bits, 14
  valcost, 14
benchmarks/__init__.py, 82
benchmarks/gscore/GnBenchLib.py, 79
benchmarks/linscore/__init__.py, 82
benchmarks/linscore/LinBench2.py, 81
benchmarks/primescore/prime-factorization.py, 82
bestConfig
  prime-factorization, 42
bestEnergy
  SimulatedAnnealing, 77
biases
  hardware.dwave.DwaveMultiplier.Multiplier, 64
BINARY
  hardware.cpu.CPUBenchmark, 30
  simpleSimulatedAnnealing.cpp, 89
binary_N
  hardware.dwave.DwaveMultiplier.Multiplier, 64
brokenConstraints
  GnBenchLib, 17
build_graph_complete
  hardware.dwave.DwaveMultiplier.Multiplier, 62

capabilities
  hardware.cpu.CPUBenchmark.CPUBenchmark, 47
  hardware.dwave.DWaveBenchmark.DWaveBenchmark, 54
  lib.QPUBenchClass.QPUBenchClass, 68
capabilitySet
  hardware.cpu.CPUBenchmark.CPUBenchmark, 49
  hardware.dwave.DWaveBenchmark.DWaveBenchmark, 55
cfa_nodes
  hardware.dwave.DwaveMultiplier.Multiplier, 62
cfa_plmt
  hardware.dwave.DwaveMultiplier.Multiplier, 64
cfa_tile_to_graph
  hardware.dwave.DwaveMultiplier.Multiplier, 62
chain_strength
  hardware.dwave.DwaveMultiplier.Multiplier, 64
Compatibility

Generated by Doxygen

hardware.dwave.DwaveMultiplier.Multiplier, 64
Compatibility_w_Real_Graph
hardware.dwave.DwaveMultiplier.Multiplier, 62
compute
hardware.cpu.CPUBenchmark.CPUBenchmark, 48
hardware.dwave.DWaveBenchmark.DWaveBenchmark, 54
lib.QPUBenchClass.QPUBenchClass, 69
computeIteration
hardware.cpu.CPUBenchmark.CPUBenchmark, 48
hardware.dwave.DWaveBenchmark.DWaveBenchmark, 54
computeRes
hardware.cpu.CPUBenchmark.CPUBenchmark, 49
hardware.dwave.DWaveBenchmark.DWaveBenchmark, 56
lib.QPUBenchClass.QPUBenchContext, 71
configbuf
SimulatedAnnealing, 77
coords
hardware.dwave.DwaveMultiplier.Multiplier, 64
couplings
hardware.dwave.DwaveMultiplier.Multiplier, 65
CPUAlgoTypes
hardware.cpu.CPUBenchmark, 30
cpuctx
hardware.cpu.CPUBenchmark.CPUBenchmark, 50
cpuDispatch
hardware.cpu.CPUBenchmark.CPUBenchmark, 50
cpusAlgo
hardware.cpu.CPUBenchmark.CPUBenchmark, 50
currentBench
hardware.cpu.CPUBenchmark.CPUBenchmark, 50
hardware.dwave.DWaveBenchmark.DWaveBenchmark, 56
lib.QPUBenchClass.QPUBenchContext, 71
currentConfig
IsingModel, 60
currentEnergy
IsingModel, 60

dat
hardware.cpu.CPUBenchmark.CPUBenchmark, 50
hardware.dwave.DWaveBenchmark.DWaveBenchmark, 56
lib.QPUBenchClass.QPUBenchContext, 71
defaultModelVal
IsingModel, 60
defMval
prime-factorization, 42
deltaTemperature
SimulatedAnnealing, 77
discardCurrent
IsingModel, 59
distVectors
lib.QPUBenchClass.QPUBenchContext, 72
dtype
hardware.cpu.CPUBenchmark, 30
dummyLin2cpu
hardware.cpu.CPUBenchmark, 26
duplicateIsing
lib.linear_algebra_ising, 36
duplicateQUBO
lib.linear_algebra_ising, 36
dwarch
hardware.dwave.DWaveBenchmark.DWaveBenchmark, 56
Dwave_Machine
hardware.dwave.DwaveMultiplier.Multiplier, 65
Dwave_Machines_dict
hardware.dwave.Dwave_machines, 31
DWaveArchTypes
hardware.dwave.DWaveBenchmark, 34
DWaveBenchmark, 15
dwaveDispatch
hardware.dwave.DWaveBenchmark.DWaveBenchmark, 56
dwctx
hardware.dwave.DWaveBenchmark.DWaveBenchmark, 56
dwembedding
hardware.dwave.DWaveBenchmark.DWaveBenchmark, 56
dwsampler
hardware.dwave.DWaveBenchmark.DWaveBenchmark, 56

edgestosummitGn
GnBenchLib, 17
Empirical_strength
hardware.dwave.Run_multiplier.Multiplier_to_Run, 67
end
GnBenchLib, 23
endingTemperature
SimulatedAnnealing, 77
energy
lib.QPUBenchClass.samplesetClass, 74
energyCost
lib.QPUBenchClass.sampledResults, 73
eq2_2_s
lib.mylinearequations, 39
eq2_2_u
lib.mylinearequations, 39
eq2_3_s
lib.mylinearequations, 40
eq2_3_u
lib.mylinearequations, 40
eq3_2_2_1_u
lib.mylinearequations, 40

eq3_2_u
lib.mylinearequations, 40
eq3_3_u
lib.mylinearequations, 40
evalGScore
GnBenchLib, 17
evalLin2Score
benchmarks.linscore.LinBench2, 12
exportQUBOdict
prime-factorization, 42

factorize_all
factorizeOnCPU.cpp, 87
hardware.cpu.CPUBenchmark, 26
factorize_all_c
factorizeOnCPU.cpp, 87
factorizeOnCPU.cpp
factorize_all, 87
factorize_all_c, 87
free_factorization, 87
ll, 87
factors
hardware.cpu.CPUBenchmark, 30
factors_nodes
hardware.dwave.DwaveMultiplier.Multiplier, 65
finalList2
prime-factorization, 43
find2bitSolutions
benchmarks.linscore.LinBench2, 12
fitSigmoid
benchmarks.linscore.LinBench2, 12
flags
hardware.cpu.CPUBenchmark, 30
flip
IsingModel, 59
float64
hardware.cpu.CPUBenchmark, 30
formatedSamples
lib.QPUBenchClass.QPUBenchContext, 72
formatSelectDwSample
hardware.dwave.DWaveBenchmark, 32
formatSelectSample
hardware.cpu.CPUBenchmark, 26
free_factorization
factorizeOnCPU.cpp, 87

g_1_qubo
lib.mylinearequations, 40
g_1_simple_qubo
lib.mylinearequations, 41
generate_embedding
hardware.dwave.DWaveBenchmark, 32
generateAB
benchmarks.linscore.LinBench2, 12
generateCandidateConfiguration
IsingModel, 59
generateXrandom
benchmarks.linscore.LinBench2, 13
get_lib_path
hardware.cpu.CPUBenchmark, 27
getBestEnergy
SimulatedAnnealing, 76
getCurrentConfig
IsingModel, 59
SimulatedAnnealing, 76
getCurrentEnergy
IsingModel, 59
getEnergy
IsingModel, 59
getHforDimod
lib.linear_algebra_ising, 37
getHQUBOforDimod
lib.linear_algebra_ising, 37
getJforDimod
lib.linear_algebra_ising, 37
getPreviousEnergy
IsingModel, 59
GnBenchLib, 15
A, 21
A2_2d, 21
A2_3d, 22
brokenConstraints, 17
edgestosummitGn, 17
end, 23
evalGScore, 17
GnMISadjacency, 17
GnQUBO, 18
gnScore, 18
h, 23
hammingDist, 18
J, 23
lowestBrokenConstraints, 19
lowestCost, 19
lowestHammingDist, 19
max, 23
maxValH, 19
myl, 23
myQ, 23
N, 23
n, 23
numval, 23
pseudosample, 23
redux2, 24
redux3, 24
root, 24
scoreRedux, 20
simpIsingh, 24
simpIsingJ, 24
simpleQUBO, 20
simplification_score, 20
simpQ, 24
successProbaGn, 21
tbc, 24
val, 24
GnMISadjacency
GnBenchLib, 17
GnQUBO

GnBenchLib, 18
gnScore
GnBenchLib, 18

h
GnBenchLib, 23
H_range
hardware.dwave.DwaveMultiplier.Multiplier, 65
hammingDist
GnBenchLib, 18
hardware, 24
hardware.cpu, 25
hardware.cpu.CPUBenchmark, 25
argtypes, 30
BINARY, 30
CPUAlgoTypes, 30
dtype, 30
dummyLin2cpu, 26
factorize_all, 26
factors, 30
flags, 30
float64, 30
formatSelectSample, 26
get_lib_path, 27
int32, 30
lib, 30
lib_path, 30
load_library, 27
lowestSamples, 27
postprocessLin2, 28
postprocessPrime, 28
preprocessLin2, 28
preprocessPrime, 28
restype, 31
root, 31
runFactorizeCPU, 28
runSAmt_py, 29
selectCPUheuristic, 29
setGnSARegister, 29
SPIN, 31
hardware.cpu.CPUBenchmark.CPUBenchmark, 45
__init__, 47
add_arguments, 47
capabilities, 47
capabilitySet, 49
compute, 48
computeIteration, 48
computeRes, 49
cpuctx, 50
cpuDispatch, 50
cpusAlgo, 50
currentBench, 50
dat, 50
is_available, 48
isInit, 50
loginfile, 50
myCompute, 50
myPostprocess, 50
myPreprocess, 50
nIter, 51
nqubits, 51
postprocess, 48
postprocessIteration, 48
postprocessRes, 51
preprocess, 48
preprocessIteration, 49
preprocessRes, 51
preprocessRes2, 51
selectBenchmark, 49
hardware.dwave, 31
hardware.dwave.Dwave_machines, 31
Dwave_Machines_dict, 31
hardware.dwave.DWaveBenchmark, 31
DWaveArchTypes, 34
formatSelectDwSample, 32
generate_embedding, 32
runDwave, 32
sampleSetDwave, 33
selectDWaveQPU, 33
setGnDwaveRegister, 33
setLin2DwaveRegister, 33
hardware.dwave.DWaveBenchmark.DWaveBenchmark, 52
__init__, 53
add_arguments, 54
capabilities, 54
capabilitySet, 55
compute, 54
computeIteration, 54
computeRes, 56
currentBench, 56
dat, 56
dwarch, 56
dwaveDispatch, 56
dwctx, 56
dwembedding, 56
dwsampler, 56
is_available, 54
isInit, 56
loginfile, 57
myCompute, 57
myPostprocess, 57
myPreprocess, 57
nIter, 57
nqubits, 57
postprocess, 54
postprocessIteration, 55
postprocessRes, 57
preprocess, 55
preprocessIteration, 55
preprocessRes, 57
selectBenchmark, 55
hardware.dwave.DwaveMultiplier, 34
hardware.dwave.DwaveMultiplier.Multiplier, 61
__init__, 62
biases, 64
binary_N, 64

build_graph_complete, 62
cfa_nodes, 62
cfa_plmt, 64
cfa_tile_to_graph, 62
chain_strength, 64
Compatibility, 64
Compatibility_w_Real_Graph, 62
coords, 64
couplings, 65
Dwave_Machine, 65
factors_nodes, 65
H_range, 65
inputs_to_fix, 65
J_range, 65
make_nodes, 63
make_subgraph, 63
N, 65
n, 65
n_col, 65
n_row, 65
offset, 66
plot_multiplier, 63
pos, 66
read_data, 63
sizes, 66
tile_0, 66
topologie_graph, 66
hardware.dwave.Run_multiplier, 34
preDwaveMultiplier, 35
returnDwavePrime, 35
save_dwave_txt, 35
useDwaveMultiplier, 35
hardware.dwave.Run_multiplier.Multiplier_to_Run, 66
__init__, 67
Empirical_strength, 67
hardware/__init__.py, 82
hardware/cpu/__init__.py, 82
hardware/cpu/CPUBenchmark.py, 82
hardware/dwave/__init__.py, 82
hardware/dwave/Dwave_machines.py, 84
hardware/dwave/DWaveBenchmark.py, 84
hardware/dwave/DwaveMultiplier.py, 85
hardware/dwave/Run_multiplier.py, 85
hardware/src/factorizeOnCPU.cpp, 86
hardware/src/simpleSimulatedAnnealing.cpp, 87
HobofyPrime
prime-factorization, 42

ILp
lib.linear_algebra_ising, 38
info
lib.QPUBenchClass.sampledResults, 73
initCoupling
SimulatedAnnealing, 76
initModel
IsingModel, 59
initRandomConfig
IsingModel, 59
initTemperature
SimulatedAnnealing, 77
inputs_to_fix
hardware.dwave.DwaveMultiplier.Multiplier, 65
int32
hardware.cpu.CPUBenchmark, 30
Iproblem
lib.linear_algebra_ising, 38
is_available
hardware.cpu.CPUBenchmark.CPUBenchmark, 48
hardware.dwave.DWaveBenchmark.DWaveBenchmark, 54
lib.QPUBenchClass.QPUBenchClass, 69
IsingModel, 58
currentConfig, 60
currentEnergy, 60
defaultModelVal, 60
discardCurrent, 59
flip, 59
generateCandidateConfiguration, 59
getCurrentConfig, 59
getCurrentEnergy, 59
getEnergy, 59
getPreviousEnergy, 59
initModel, 59
initRandomConfig, 59
IsingModel, 58
isingSize, 60
isingType, 60
myh, 60
myJ, 60
previousConfig, 60
previousEnergy, 61
selectCurrent, 60
IsingOutput
lib.linear_algebra_ising, 37
isingSize
IsingModel, 60
IsingtoQUBO
lib.linear_algebra_ising, 37
isingType
IsingModel, 60
isInit
hardware.cpu.CPUBenchmark.CPUBenchmark, 50
hardware.dwave.DWaveBenchmark.DWaveBenchmark, 56
Isize
SimulatedAnnealing, 77

J
GnBenchLib, 23
J_range
hardware.dwave.DwaveMultiplier.Multiplier, 65

lib, 36
hardware.cpu.CPUBenchmark, 30
lib.linear_algebra_ising, 36
A, 38

B, 38
duplicateIsing, 36
duplicateQUBO, 36
getHforDimod, 37
getHQUBOforDimod, 37
getJforDimod, 37
ILp, 38
Iproblem, 38
IsingOutput, 37
IsingtoQUBO, 37
maxValIsing, 37
maxValQUBO, 37
numbit, 38
Q2, 38
Qg, 39
QLp, 39
Qproblem, 39
QUBOfy, 37
QUBOoutput, 38
QUBOtoIsing, 38
sig, 39
lib.mylinearequations, 39
eq2_2_s, 39
eq2_2_u, 39
eq2_3_s, 40
eq2_3_u, 40
eq3_2_2_1_u, 40
eq3_2_u, 40
eq3_3_u, 40
g_1_qubo, 40
g_1_simple_qubo, 41
lib.QPUBenchClass, 41
lib.QPUBenchClass.QPUBenchClass, 68
add_arguments, 68
capabilities, 68
compute, 69
is_available, 69
postprocess, 69
preprocess, 69
selectBenchmark, 69
lib.QPUBenchClass.QPUBenchClassCapability, 70
name, 70
supports, 70
lib.QPUBenchClass.QPUBenchContext, 71
arch, 71
computeRes, 71
currentBench, 71
dat, 71
distVectors, 72
formatedSamples, 72
nIter, 72
proba, 72
probaDist, 72
subarch, 72
target, 72
lib.QPUBenchClass.sampledResults, 73
energyCost, 73
info, 73
num, 73
numQubits, 73
samples, 73
totNum, 74
lib.QPUBenchClass.samplesetClass, 74
energy, 74
occurences, 74
vecVal, 74
lib/__init__.py, 82
lib/linear_algebra_ising.py, 89
lib/mylinearequations.py, 90
lib/QPUBenchClass.py, 90
lib_path
hardware.cpu.CPUBenchmark, 30
Lin2Prob
benchmarks.linscore.LinBench2, 13
LinBench2, 41
LinProb
benchmarks.linscore.LinBench2, 13
list34
prime-factorization, 43
listupdate
prime-factorization, 42
ll
factorizeOnCPU.cpp, 87
load_library
hardware.cpu.CPUBenchmark, 27
loginfile
hardware.cpu.CPUBenchmark.CPUBenchmark, 50
hardware.dwave.DWaveBenchmark.DWaveBenchmark, 57
lowestBrokenConstraints
GnBenchLib, 19
lowestCost
GnBenchLib, 19
lowestHammingDist
GnBenchLib, 19
lowestSamples
hardware.cpu.CPUBenchmark, 27

main
BACQBenchmark, 9
benchmarks.linscore.LinBench2, 13
make_nodes
hardware.dwave.DwaveMultiplier.Multiplier, 63
make_subgraph
hardware.dwave.DwaveMultiplier.Multiplier, 63
max
GnBenchLib, 23
maxValH
GnBenchLib, 19
maxValIsing
lib.linear_algebra_ising, 37
maxValQUBO
lib.linear_algebra_ising, 37
ModelType
simpleSimulatedAnnealing.cpp, 89
modpath

BACQBenchmark, 10
module
BACQBenchmark, 10
myCompute
hardware.cpu.CPUBenchmark.CPUBenchmark, 50
hardware.dwave.DWaveBenchmark.DWaveBenchmark, 57
myh
IsingModel, 60
prime-factorization, 43
myIsing
SimulatedAnnealing, 77
myJ
IsingModel, 60
prime-factorization, 43
myl
GnBenchLib, 23
mylist2
prime-factorization, 43
mylist3
prime-factorization, 43
mylist4
prime-factorization, 43
myPostprocess
hardware.cpu.CPUBenchmark.CPUBenchmark, 50
hardware.dwave.DWaveBenchmark.DWaveBenchmark, 57
myPreprocess
hardware.cpu.CPUBenchmark.CPUBenchmark, 50
hardware.dwave.DWaveBenchmark.DWaveBenchmark, 57
myQ
GnBenchLib, 23

N
GnBenchLib, 23
hardware.dwave.DwaveMultiplier.Multiplier, 65
n
GnBenchLib, 23
hardware.dwave.DwaveMultiplier.Multiplier, 65
n_col
hardware.dwave.DwaveMultiplier.Multiplier, 65
n_row
hardware.dwave.DwaveMultiplier.Multiplier, 65
name
lib.QPUBenchClass.QPUBenchClassCapability, 70
newlist2
prime-factorization, 43
nIter
hardware.cpu.CPUBenchmark.CPUBenchmark, 51
hardware.dwave.DWaveBenchmark.DWaveBenchmark, 57
lib.QPUBenchClass.QPUBenchContext, 72
nqubits
hardware.cpu.CPUBenchmark.CPUBenchmark, 51
hardware.dwave.DWaveBenchmark.DWaveBenchmark, 57
num
lib.QPUBenchClass.sampledResults, 73
numbit
lib.linear_algebra_ising, 38
numQubits
lib.QPUBenchClass.sampledResults, 73
numSwipe
SimulatedAnnealing, 77
numval
GnBenchLib, 23

occurences
lib.QPUBenchClass.samplesetClass, 74
offset
hardware.dwave.DwaveMultiplier.Multiplier, 66
OMP_PARALLEL
simpleSimulatedAnnealing.cpp, 88

plot_multiplier
hardware.dwave.DwaveMultiplier.Multiplier, 63
pos
hardware.dwave.DwaveMultiplier.Multiplier, 66
postprocess
hardware.cpu.CPUBenchmark.CPUBenchmark, 48
hardware.dwave.DWaveBenchmark.DWaveBenchmark, 54
lib.QPUBenchClass.QPUBenchClass, 69
postprocessIteration
hardware.cpu.CPUBenchmark.CPUBenchmark, 48
hardware.dwave.DWaveBenchmark.DWaveBenchmark, 55
postprocessLin2
hardware.cpu.CPUBenchmark, 28
postprocessPrime
hardware.cpu.CPUBenchmark, 28
postprocessRes
hardware.cpu.CPUBenchmark.CPUBenchmark, 51
hardware.dwave.DWaveBenchmark.DWaveBenchmark, 57
preDwaveMultiplier
hardware.dwave.Run_multiplier, 35
preprocess
hardware.cpu.CPUBenchmark.CPUBenchmark, 48
hardware.dwave.DWaveBenchmark.DWaveBenchmark, 55
lib.QPUBenchClass.QPUBenchClass, 69
preprocessIteration
hardware.cpu.CPUBenchmark.CPUBenchmark, 49
hardware.dwave.DWaveBenchmark.DWaveBenchmark, 55

preprocessLin2
hardware.cpu.CPUBenchmark, 28
preprocessPrime
hardware.cpu.CPUBenchmark, 28
preprocessRes
hardware.cpu.CPUBenchmark.CPUBenchmark, 51
hardware.dwave.DWaveBenchmark.DWaveBenchmark, 57
preprocessRes2
hardware.cpu.CPUBenchmark.CPUBenchmark, 51
previousConfig
IsingModel, 60
previousEnergy
IsingModel, 61
prime-factorization, 41
bestConfig, 42
defMval, 42
exportQUBOdict, 42
finalList2, 43
HobofyPrime, 42
list34, 43
listupdate, 42
myh, 43
myJ, 43
mylist2, 43
mylist3, 43
mylist4, 43
newlist2, 43
quadratize, 42
reducePolynomial, 43
tofactorize, 44
totvar, 44
valuedlist, 44
proba
lib.QPUBenchClass.QPUBenchContext, 72
probaDist
lib.QPUBenchClass.QPUBenchContext, 72
pseudosample
GnBenchLib, 23

Q2
lib.linear_algebra_ising, 38
Qg
lib.linear_algebra_ising, 39
QLp
lib.linear_algebra_ising, 39
Qproblem
lib.linear_algebra_ising, 39
QPUBenchClass, 44
quadratize
prime-factorization, 42
QUBOfy
lib.linear_algebra_ising, 37
QUBOLin2Bench
benchmarks.linscore.LinBench2, 14
QUBOoutput
lib.linear_algebra_ising, 38
QUBOtoIsing
lib.linear_algebra_ising, 38

read_data
hardware.dwave.DwaveMultiplier.Multiplier, 63
reducePolynomial
prime-factorization, 43
redux2
GnBenchLib, 24
redux3
GnBenchLib, 24
restype
hardware.cpu.CPUBenchmark, 31
result
SimulatedAnnealing, 76
returnDwavePrime
hardware.dwave.Run_multiplier, 35
root
benchmarks.linscore.LinBench2, 15
GnBenchLib, 24
hardware.cpu.CPUBenchmark, 31
run
SimulatedAnnealing, 76
runDwave
hardware.dwave.DWaveBenchmark, 32
runFactorizeCPU
hardware.cpu.CPUBenchmark, 28
runSAmt_py
hardware.cpu.CPUBenchmark, 29

samples
lib.QPUBenchClass.sampledResults, 73
sampleSetDwave
hardware.dwave.DWaveBenchmark, 33
save_dwave_txt
hardware.dwave.Run_multiplier, 35
scale
SimulatedAnnealing, 77
scoreRedux
GnBenchLib, 20
selectBenchmark
hardware.cpu.CPUBenchmark.CPUBenchmark, 49
hardware.dwave.DWaveBenchmark.DWaveBenchmark, 55
lib.QPUBenchClass.QPUBenchClass, 69
selectCPUheuristic
hardware.cpu.CPUBenchmark, 29
selectCurrent
IsingModel, 60
selectDWaveQPU
hardware.dwave.DWaveBenchmark, 33
setCurrentAsBest
SimulatedAnnealing, 76
setGnDwaveRegister
hardware.dwave.DWaveBenchmark, 33
setGnSARegister
hardware.cpu.CPUBenchmark, 29
setInitialTemperature

SimulatedAnnealing, 76
setLin2DwaveRegister
hardware.dwave.DWaveBenchmark, 33
setScale
SimulatedAnnealing, 76
setSwipeNumber
SimulatedAnnealing, 76
sig
lib.linear_algebra_ising, 39
simpIsingh
GnBenchLib, 24
simpIsingJ
GnBenchLib, 24
simpleQUBO
GnBenchLib, 20
simpleSimulatedAnnealing.cpp
BINARY, 89
ModelType, 89
OMP_PARALLEL, 88
SPIN, 89
uniform, 89
uniformDist, 89
simplification_score
GnBenchLib, 20
simpQ
GnBenchLib, 24
SimulatedAnnealing, 75
bestEnergy, 77
configbuf, 77
deltaTemperature, 77
endingTemperature, 77
getBestEnergy, 76
getCurrentConfig, 76
initCoupling, 76
initTemperature, 77
Isize, 77
myIsing, 77
numSwipe, 77
result, 76
run, 76
scale, 77
setCurrentAsBest, 76
setInitialTemperature, 76
setScale, 76
setSwipeNumber, 76
SimulatedAnnealing, 75
sizes
hardware.dwave.DwaveMultiplier.Multiplier, 66
SPIN
hardware.cpu.CPUBenchmark, 31
simpleSimulatedAnnealing.cpp, 89
subarch
lib.QPUBenchClass.QPUBenchContext, 72
successProbaGn
GnBenchLib, 21
successProbaLin2
benchmarks.linscore.LinBench2, 14
supports
lib.QPUBenchClass.QPUBenchClassCapability, 70

target
lib.QPUBenchClass.QPUBenchContext, 72
tbc
GnBenchLib, 24
tile_0
hardware.dwave.DwaveMultiplier.Multiplier, 66
tofactorize
prime-factorization, 44
topologie_graph
hardware.dwave.DwaveMultiplier.Multiplier, 66
totNum
lib.QPUBenchClass.sampledResults, 74
totvar
prime-factorization, 44
truncate2bits
benchmarks.linscore.LinBench2, 14

uniform
simpleSimulatedAnnealing.cpp, 89
uniformDist
simpleSimulatedAnnealing.cpp, 89
useDwaveMultiplier
hardware.dwave.Run_multiplier, 35

val
GnBenchLib, 24
valcost
benchmarks.linscore.LinBench2, 14
valuedlist
prime-factorization, 44
vecVal
lib.QPUBenchClass.samplesetClass, 74